



UNIVERSITÀ
DI CAMERINO

Scuola di scienze e tecnologie

Corso di laurea in informatica

Studio dei protocolli DSA ed ECDSA e loro implementazioni nei linguaggi C e Java

tesi di laurea

candidato:

Marco Cocilova

codice matricola: 113965

Marco Cocilova

relatore:

Prof. Michele Loreti

Michele Loreti

A/A 2023-24

Indice

Introduzione.....	4
1 Fondamenti di crittografia	6
1.1 Principi e applicazioni della crittografia	6
1.1.1 Proprietà dell'elemento più debole	7
1.1.2 Ambiente ostile	8
1.2 Crittografia simmetrica e asimmetrica	9
1.2.1 Cifratura reversibile e non reversibile.....	9
1.3 Firma digitale	11
1.3.1 Firma digitale come l'elemento più forte.....	11
1.3.2 Chiavi poco variabili	12
2 Concetti di matematica.....	13
2.1 Matematica modulare	13
2.2 Problema del logaritmo discreto (teoria generale).....	16
Problema del logaritmo discreto su gruppi moltiplicativi	16
Problema del logaritmo discreto su gruppi di punti di una curva ellittica	17
3 Algoritmi di firma digitale trattati.....	19
3.1 Digital Signature Algorithm (DSA)	19
3.2 Elliptic Curve Digital Signature Algorithm (ECDSA).....	21
3.2.1 Elliptic Curve Cryptography (ECC)	22
3.2.2 Ordine di un punto su una curva ellittica.....	23
Coordinate proiettive	23
3.3 DSA ed ECDSA a confronto	25
4 DSA ed ECDSA: implementazione degli algoritmi	26
4.1 Oracle (JAVA).....	26
4.1.1 Java Security.....	26
Personalizzazione dei parametri	27
Metodo getInstance(...).....	27
Classe BigInteger	28
Interfacce di marcatura AlgorithmParameterSpec	28
Classe astratta KeyPairGenerator	29
Classe KeyFactory.....	30
Classe astratta Signature	30
4.1.2 NativeDS	32
Classe astratta ParametersCalculator	32
Classe astratta ParametersExtractor	33

Classe OperationsManager	33
Classe SecureRandomNumberGenerator	34
4.1.3 Implementazioni esterne	34
4.1.4 Configurazioni per DSA ed ECDSA in Java	35
Configurazioni di avvio	35
4.2 Oryx Embedded (C)	37
4.2.1 Modifiche apportate	37
4.2.2 Libreria CycloneCrypto	37
Files mpi	37
Files dsa	39
Files ecdsa ed ec	40
4.2.3 Libreria Common	42
4.2.4 Files di configurazione esterni	42
4.2.5 Configurazioni per DSA ed ECDSA in C	43
Configurazioni di avvio	43
Serializzazione della firma	44
Conclusione	46

Introduzione

La sicurezza delle comunicazioni digitali è divenuta uno degli aspetti cruciali nella moderna società tecnologica. Tra i meccanismi chiave che garantiscono l'autenticità e l'integrità delle informazioni scambiate vi sono le firme digitali, strumenti fondamentali nella crittografia asimmetrica. Due degli algoritmi più significativi in questo ambito sono il **Digital Signature Algorithm (DSA)** e la sua variante basata su curve ellittiche, l'**Elliptic Curve Digital Signature Algorithm (ECDSA)**.

Questa tesi si propone di analizzare in dettaglio entrambi gli algoritmi, fornendo un confronto dal punto di vista matematico e prestazionale. In particolare, verranno esaminati i contesti crittografici sottostanti a **DSA** ed **ECDSA**, con un focus sul problema del logaritmo discreto e sulle curve ellittiche, per poi passare a una valutazione dell'efficienza e della sicurezza offerta da ciascuno di essi.

Oltre all'analisi teorica, sono stati implementati e testati entrambi gli algoritmi allo scopo di mostrare la fattibilità di configurazioni personalizzate di queste due firme tramite l'ausilio di apposite librerie nei due linguaggi di programmazione **C** e **Java**.

La struttura della tesi si articola in tre principali sezioni: una prima parte dedicata alla teoria matematica alla base degli algoritmi, una seconda parte comparativa in termini di efficienza e sicurezza e, infine, una sezione incentrata sull'implementazione del codice, il cui link è disponibile nella bibliografia.

Le implementazioni proposte non hanno lo scopo di introdurre nuove scoperte o utilizzi innovativi degli algoritmi, ma forniscono nuove funzioni nella libreria **CycloneCrypto** per **C** e una nuova libreria che integra la **Java.Security**, permettendo l'uso di parametri personalizzati tramite il calcolo nativo.

Accenno Storico

L'algoritmo **DSA**, che ha avuto il suo picco di diffusione nel **1991**, è attualmente poco utilizzato e messo in secondo piano dall'algoritmo **RSA**, oggi considerato uno standard tra grandi aziende ed enti governativi.

Inizialmente **DSA** è stato l'algoritmo più diffuso perché, siccome **RSA** era ancora sotto brevetto, non c'era una significativa concorrenza, ma dopo la scadenza del brevetto **RSA** nel **2000**, **DSA** ha avuto un declino tanto rapido quanto la sua ascesa.

Il fatto che **RSA** viene preferito rispetto a **DSA** pur avendo lo stesso livello di sicurezza è dovuto a motivazioni di natura pratica e di efficienza in un contesto specifico.

La versatilità di **RSA** è dovuta al fatto che questo algoritmo può essere utilizzato anche per la cifratura reversibile, ma anche al fatto che è più semplice da implementare.

Per quanto riguarda l'efficienza invece, **DSA** è più efficiente per quanto riguarda la generazione della firma, ma non per la verifica. Questo fattore lo rende meno preferibile in un ambiente in cui sono necessari numerosi controlli, come avviene nelle comunicazioni in rete.

Nonostante **DSA** sia stato accantonato, durante il suo periodo di popolarità sono state ideate delle varianti, tra cui l'algoritmo **ECDSA**.

Questa **variante di DSA** sta attualmente avendo molto successo in quanto garantisce lo stesso

livello di sicurezza di una firma **DSA** ed **RSA** utilizzando delle chiavi molto più piccole, rendendo questo algoritmo un'eccellente opzione per ambienti con minore potenza di calcolo come gli ambienti firmware e mobile.

1 Fondamenti di crittografia

La crittografia è nata come disciplina dedicata principalmente alla cifratura, ovvero l'atto di rendere illeggibile il contenuto di un dato in maniera reversibile e senza perdita di informazioni.

Nel tempo, il campo si è ampliato enormemente, arrivando a comprendere molte altre funzioni di sicurezza, come l'autenticazione, le firme digitali, e la protezione della privacy.

Oggi la crittografia abbraccia un'ampia gamma di competenze che includono l'analisi statistica, la teoria dei numeri, l'informatica teorica ed anche questioni legali come il diritto civile e la privacy.

Per costruire sistemi crittografici ben fatti servono una buona cultura sia in ambito matematico che informatico, ma anche una buona dose di mentalità e pensiero laterale, adatti a riflettere con creatività sui problemi.

La crittografia è un campo complesso, in cui anche i più esperti progettano sistemi che qualche anno dopo vengono violati; si tratta di casi comuni che non destano sorpresa.

La proprietà dell'elemento più debole e l'ambiente ostile (che verranno spiegati in dettaglio più avanti) cospirano nel rendere il compito dei tecnici di crittografia, e della sicurezza in generale, molto difficile.

Un altro problema significativo è la mancanza di test efficaci; infatti non esiste un modo noto per testare se un sistema è sicuro.

Nella comunità che si occupa di ricerca nella sicurezza e nella crittografia, per esempio, si cerca di rendere pubblici i sistemi progettati. In questo modo altri esperti possono esaminare e risolvere le vulnerabilità della sicurezza, ma nonostante ciò le verifiche svolte da conferenze e riviste non sono sufficienti per individuare tutti i potenziali problemi di sicurezza.

Anche se molti occhi esperti esaminano un'architettura, può darsi che esistano lacune che si manifesteranno soltanto dopo anni.

1.1 Principi e applicazioni della crittografia

Nell'ambito della crittografia esistono dei principi che definiscono le caratteristiche essenziali richieste in ogni sistema crittografico. Tali principi contribuiscono ad assicurare completezza ed affidabilità.

Di seguito sono riportati questi principi:

Confidenzialità

Le informazioni trasmesse devono essere **accessibili solo da chi è autorizzato**.

Questa proprietà viene garantita sia con l'ausilio di algoritmi di cifratura reversibili (come l'AES) sia da algoritmi non reversibili (come l'RSA).

Una cifratura reversibile può essere utilizzata in sistemi in cui l'informazione inviata dovrà essere effettivamente letta in un secondo momento, dovendo quindi fornire la possibilità di riconvertire il testo cifrato nel testo in chiaro. Tale sistema è adoperato da praticamente tutti i sistemi che richiedono uno scambio di informazioni, eseguiti tramite canali poco sicuri come il web.

Una cifratura non reversibile è invece un'opzione più adatta per immagazzinare i dati senza fornire la possibilità di ottenerne le informazioni dai dati immagazzinati.

Tale soluzione è strettamente raccomandata in strutture che utilizzano i dati archiviati con il solo scopo di verifica, come avviene ad esempio per le autenticazioni tramite password. Di fatto è un'ottima convenzione per il database di un server di conservare le password degli utenti in formato hash, comparando i testi cifrati per evitare di esporre le informazioni in chiaro. Ciò aggiunge un rilevante livello di sicurezza in caso di attacchi al database.

Integrità

Deve esserci una **garanzia che le informazioni ricevute non siano state manomesse** durante il trasporto.

Questa proprietà viene garantita tramite l'accostamento del formato non-reversibilmente cifrato (come l'hash) col messaggio inviato.

In questa maniera, non conoscendo l'algoritmo o la chiave di cifratura, un attaccante che ha manomesso il messaggio non è in grado di generare un hash valido per ingannare il destinatario. Modificando il messaggio originale si causa la generazione di un hash differente da quello iniziale, permettendo di riconoscere un falso.

Controllo degli Accessi

Un sistema deve includere un protocollo di **riconoscimento degli utenti e di gestione delle autorizzazioni** sulle operazioni all'interno del sistema.

Questa proprietà deve essere garantita implicitamente o esplicitamente in qualunque sistema sicuro. Il riconoscimento può variare dall'uso di nome e password, utilizzo di un'impronta digitale o un qualunque altro metodo identificabile come sicuro.

Autenticazione

Deve essere possibile **risalire all'identità della persona o dell'utente** che ha inviato le informazioni. Questa proprietà può essere garantita con un sistema di accesso o con altri metodi di riconoscimento come l'IP del dispositivo.

Non-Ripudio

Le informazioni inviate devono essere inequivocabilmente **correlate con l'identità reale** della persona che le ha inviate, rendendo impossibile la negazione dell'autenticità delle informazioni (salvo denuncia alla compromissione del sistema).

Questa proprietà viene garantita da sistemi di riconoscimento sicuri personali come una firma digitale relativa all'identità della persona.

1.1.1 Proprietà dell'elemento più debole

La crittografia, così come una serratura applicata ad una porta, ha senso di essere utilizzata soltanto su sistemi più ampi che includono il metodo di sicurezza, i dati da proteggere e le persone che lo adopereranno.

Per quanto fondamentale, la **crittografia è soltanto una piccola parte di un sistema di sicurezza** molto più ampio.

La crittografia è il componente del sistema avente la responsabilità di concedere l'accesso solo ad alcune persone. Questo è un aspetto molto delicato ed anche il più difficile da realizzare in un

sistema sicuro.

Per questo motivo la crittografia e i suoi elementi di contorno costituiscono un punto di attacco naturale per qualsiasi sistema di sicurezza.

Ciò non significa che la crittografia sia sempre il punto debole. In alcuni casi, anche una cattiva crittografia può essere molto migliore di tutto il resto del sistema.

L'uso della crittografia non comporta automaticamente un incremento della sicurezza nel sistema se esso non è ben strutturato e presenta punti deboli, perché la sicurezza complessiva di un sistema di sicurezza equivale a quella fornita dall'elemento più debole.

Per migliorare efficacemente la sicurezza di un sistema occorre **migliorare l'elemento più debole**, ma per fare ciò è necessario conoscere quali sono gli elementi e quali sono i più deboli.

1.1.2 Ambiente ostile

L'ambiente ostile è un concetto utilizzato durante la progettazione di qualunque tipo di sistema di sicurezza, ovviamente focalizzandosi su determinati ambiti piuttosto che altri a seconda del contesto in cui esso viene realizzato.

Considerare il proprio sistema all'interno di un ambiente ostile significa valutare come ogni tipo di possibile minaccia possa intaccare la sicurezza.

Progettare un sistema di sicurezza richiede **avere ben chiaro cosa si vuole proteggere e da chi**.

Queste domande apparentemente semplici sono essenziali per stabilire le misure più appropriate da applicare.

Buoni atteggiamenti sono quello di **considerare qualunque attore** (persino gli operatori stessi) **come un possibile** attaccante e quello di **cercare una vulnerabilità in qualunque struttura ed operazione**.

Raramente un attaccante bersaglia gli elementi attesi con i metodi previsti, ma al contrario tenderà a trovare debolezze non identificate dai progettisti della sicurezza.

1.2 Crittografia simmetrica e asimmetrica

Molti sistemi crittografici utilizzano delle chiavi per generare messaggi cifrati o per firmare dei dati. E quando viene applicata una chiave da un lato, sarà necessario utilizzare una chiave anche per decifrare il messaggio o verificare la firma.

L'uso delle chiavi nella crittografia si divide in crittografia **simmetrica** e **asimmetrica**, differendo per l'uso delle chiavi.

La crittografia simmetrica adopera una chiave detta **segreta**, la quale permette a chi la possiede sia di criptare che di deciptare i dati.

Questo sistema permette un interscambio sicuro di dati all'interno di una cerchia di persone che, utilizzando la medesima chiave segreta, possono cifrare e decifrare i messaggi evitando che altre persone al di fuori del gruppo possano estrarre o immettere informazioni.

La crittografia asimmetrica utilizza una coppia di chiavi dette **privata** e **pubblica**, le quali coppie sono univocamente legate tra loro.

La chiave privata serve per cifrare i dati, mentre quella pubblica viene utilizzata per decifrarli.

Questo sistema permette solo a chi possiede la chiave privata di pubblicare i dati criptati, mentre la chiave pubblica viene utilizzata solo per estrarre le informazioni.

La peculiarità della crittografia asimmetrica di permettere solo a chi possiede una chiave privata di pubblicare le informazioni è particolarmente utile in quanto implica un'**autenticità** dell'origine dei dati. Infatti se un dato è decifrabile da una determinata chiave pubblica, allora è possibile dedurre che esso sia stato creato dalla corrispettiva chiave privata.

1.2.1 Cifratura reversibile e non reversibile

Il concetto di cifratura a chiave simmetrica e cifratura a chiave asimmetrica non è da associare col concetto di cifratura reversibile e non reversibile.

Sia la cifratura simmetrica che quella asimmetrica possono adottare sia algoritmi reversibili che non reversibili per poter essere utilizzati in situazioni specifiche.

Ufficialmente con le espressioni "cifratura simmetrica" e "cifratura asimmetrica" si fa riferimento ad algoritmi reversibili, ma per esplicitare maggiormente la distinzione tra reversibilità e non reversibilità, farò una distinzione esplicita tra "cifre reversibili" e "cifre non-reversibili":

Cifratura simmetrica reversibile:

Algoritmi come l'**Advanced Encryption Standard (AES)** vengono utilizzati per un interscambio sicuro di dati in un ambiente in cui il rischio o la gravità della compromissione della chiave segreta è sufficientemente basso e non è richiesta l'autenticazione degli utenti che trasmettono i dati.

Cifratura asimmetrica reversibile:

Algoritmi come l'**RSA** vengono utilizzati, oltre che per un agile sistema di firma digitale, per un interscambio sicuro di dati distribuendo una chiave pubblica.

Distribuire una chiave pubblica comporta un rischio molto minore che utenti non autorizzati possano ottenere la possibilità di falsificare i dati inviati, rendendo questa opzione molto adatta in

casi in cui molti utenti devono ricevere delle informazioni con la necessità di un'autenticazione del mittente.

Cifratura simmetrica non-reversibile:

Gli algoritmi di hashing come **SHA** rappresentano perfettamente il concetto di cifratura non reversibile a chiave singola.

Essi vengono adoperati per operazioni di verifica dell'integrità e dell'autenticità dei dati.

Due esempi d'uso sono:

-L'**hash di verifica** inviato insieme ad un documento, il quale viene verificato dal mittente cifrando il documento e comparandolo con l'hash associato.

-L'**archiviazione delle password** in formato hash anziché in chiaro così da prevenire un furto di informazioni. Per verificare le password inserite durante gli accessi è sufficiente cifrare la password inserita dall'utente e compararla con l'hash del database.

Cifratura asimmetrica non-reversibile:

Gli algoritmi di firma digitale come la **Digital Signature Algorithm (DSA)** sono dei buoni rappresentanti per questo concetto in quanto utilizzano coppie di chiavi per generare una firma, la quale può essere considerata una sovra specie di hashing che, oltre ad essere adoperato per la verifica dell'integrità e dell'autenticità dei dati, permette il **non-ripudio** da parte del mittente.

1.3 Firma digitale

Una firma digitale è una forma di firma elettronica che utilizza tecniche crittografiche con coppie di chiavi per garantire l'**autenticità** di un documento e l'**identità** del firmatario, nonché per garantire il **non-ripudio** del documento firmato.

Una firma digitale viene adoperata per validare documenti in modo da garantire che il mittente sia quello atteso e che il documento non sia stato compromesso dopo l'invio.

A differenza di una firma anagrafica, che deve essere uguale per ogni documento, il contenuto della firma digitale varia a seconda del documento firmato oltre che dal firmatario.

A rappresentare l'identità di una persona è un **certificato per firma digitale**, il quale contiene varie informazioni tra cui dati anagrafici, chiave pubblica, versione e algoritmi utilizzati.

Quindi, in linea di massima, l'identità di una persona è rappresentata dall'insieme di chiave privata, algoritmo di firma e i parametri impostati per l'algoritmo.

Per firmare un documento, il mittente dà in pasto all'algoritmo il documento da firmare e la propria chiave, in questo modo verrà generata una firma.

La firma generata sarà relativa univocamente sia al firmatario che al documento firmato, permettendo quindi di essere una prova giuridica sufficiente in cause legali, a patto che essa rispetti i requisiti di legge.

Chiunque volesse verificare la firma avrà bisogno di una copia della chiave pubblica del firmatario, univoca per ogni chiave privata, l'algoritmo utilizzato, i parametri utilizzati e il documento da verificare.

1.3.1 Firma digitale come l'elemento più forte

La firma digitale è considerata uno degli elementi più forti in un sistema crittografico perché si basa su algoritmi robusti che sfruttano problemi matematici complessi come il **problema del logaritmo discreto**, noti per la loro resistenza agli attacchi crittoanalitici.

Questi algoritmi sono progettati per garantire l'integrità e l'autenticità dei dati firmati, rendendo estremamente difficile, se non impossibile, per un attaccante compromettere una firma senza conoscere la chiave privata corrispondente.

L'uso di un sistema di autenticazione a chiave pubblica è molto agile e sicuro in quanto è possibile distribuire la chiave necessaria per le verifiche senza doversi preoccupare né che essa venga diffusa eccessivamente né di garantire opportune distruzioni delle vecchie chiavi.

La probabilità che la stessa chiave pubblica possa essere riutilizzata come gemella per un'altra coppia di chiavi è estremamente vicina allo zero, inoltre la lunghezza delle chiavi viene periodicamente incrementata, rendendo inutilizzabili tutte le vecchie chiavi di lunghezza inferiore. Questi fattori rendono sicuro lasciare che le vecchie chiavi pubbliche rimangano di pubblico dominio dopo il cambio della chiave privata utilizzata.

1.3.2 Chiavi poco variabili

Il calcolo delle variabili utilizzate per generare una firma digitale è molto oneroso, specialmente utilizzando valori estremamente grandi come numeri di **1024 bit**, che appartengono all'ordine di grandezza di 10^{308} , rendendo quindi improponibile cambiare continuamente i valori utilizzati per generare le due chiavi.

Per questa ragione esistono dei valori standard precalcolati disponibili per i sistemi di firma digitale più diffusi.

Utilizzare parametri precalcolati è conveniente in sistemi ad alte prestazioni che generano continuamente nuove connessioni e gestire un gran numero di chiavi.

L'utilizzo dei parametri standard permette di risparmiare tempo e risorse necessari per estrarre e computare i valori, ma potrebbe causare una riduzione della sicurezza nel caso di attaccanti che hanno studiato tali parametri.

Utilizzare valori precalcolati potrebbe sembrare un buon motivo per sviluppare un sistema che necessita di una continua generazione di nuove chiavi, e lo è, però non sempre si tratta di un'operazione necessaria e conveniente.

Utilizzare numerose chiavi diverse al fine di proteggersi da attacchi crittoanalitici è effettivamente un metodo efficace.

Avendo meno tempo per analizzare le varie firme utilizzate, ci sono meno probabilità che un attaccante riesca ad ottenere la chiave privata prima che essa venga sostituita, inoltre, sempre a causa dei cambi più frequenti, il danno causato da un'eventuale compromissione di una chiave sarebbe limitato dall'intervallo di tempo in cui tale chiave viene utilizzata.

Ciò che potrebbe rendere inappropriato questo metodo è il fatto che la firma digitale è solitamente l'elemento più forte di un sistema e, data la proprietà dell'elemento più debole, fortificare ulteriormente l'elemento più forte comporta solamente un minimo aumento della sicurezza, dovendo comunque affrontare le complicazioni causate dall'aumento della complessità della manutenzione del sistema.

In questo caso specifico il ricorrente cambio della chiave privata richiederebbe varie procedure che aumentano l'onere della manutenzione: tra queste vi sono l'implementazione di un sistema di gestione delle chiavi, la progettazione di un protocollo per aggiornare o eliminare efficacemente le vecchie chiavi già distribuite, sviluppare un'eventuale funzionalità di retro compatibilità o di rifirma dei documenti e un aumento di controlli atti alla prevenzione di errori umani durante le procedure di aggiornamento delle chiavi.

In sintesi è fondamentale bilanciare un sistema troppo statico che fa affidamento sulla presunta inviolabilità dei suoi elementi e un sistema che richiede un'eccessivo carico di risorse e di attenzioni per un incremento minimo della sicurezza complessiva.

2 Concetti di matematica

In questa sezione vengono spiegati i concetti che sono alla base delle operazioni matematiche eseguite dagli algoritmi di firma digitale trattati in questo documento.

Le spiegazioni esposte coprono solamente i concetti necessari per la comprensione dei due algoritmi **DSA** ed **ECDSA** trattati in questo documento.

2.1 Matematica modulare

La matematica modulare consiste nello svolgere calcoli ed utilizzare le proprietà dei numeri in un **sistema di numeri ricorrenti** definito dal modulo utilizzato. Ovvero un sistema nel quale i numeri, una volta raggiunto un determinato valore, vengono riportati al valore neutro **0**, dando origine ad un **ciclo ricorrente**.

Dato un modulo **n**, si definisce un intervallo **[0, n-1]** che compone il **gruppo ciclico** in cui avverranno le operazioni. Tutti i valori devono essere definiti all'interno dell'**intervallo del loro gruppo ciclico**.

Questo processo rende le operazioni "cicliche": ad esempio, se si parte da **0** e si continua ad incrementare il numero di una unità, una volta arrivati ad eguagliare il modulo, tale valore tornerà ad essere uguale a **0**.

Ogni operazione, come somma o moltiplicazione, viene effettuata all'interno di questo intervallo, e i risultati che escono da esso vengono riportati all'interno tramite l'operazione di **riduzione modulare**. Il quale processo consiste nell'eseguire una divisione e considerare il resto come il risultato del calcolo.

Esempi di operazioni modulari:

$4 + 2 \pmod{5} = 1$ (perché 6 diviso 5 dà resto 1)

$3 - 5 \pmod{7} = 5$ (perché $-2 / 7 = -1$ resto 5)

$27 \pmod{10} = 7$ (perché 27 ridotto modulo 10 dà resto 7)

Questa ciclicità implica che un'equazione modulare può avere infinite soluzioni ricorrenti. Ad esempio, l'equazione **$2+x \pmod{7} = 5$** ha **$x=3$** come soluzione principale, ma sono soluzioni valide anche **$x = \{10, 17, 24, \dots\}$** (cioè **$3+7k$** , dove **k** è un intero).

congruenza

L'operazione fondamentale nella matematica modulare è la **congruenza**, indicata dal simbolo ' $\equiv$ '.

Dati due numeri interi **a** e **b** ed un modulo intero positivo **n**, i due interi **a** e **b** si dicono congruenti tra loro se e solo se il resto della divisione **a/n** è uguale al resto della divisione **b/n**.

La congruenza **$17 \equiv 22 \pmod{5}$** è vera perché sia **17** che **22** portano resto **2** se divisi per **5**, quindi si dicono **congruenti modulo 5**.

Inverso moltiplicativo modulare

L'inverso moltiplicativo di un valore **k**, espresso come **k^{-1}** , è il valore che se moltiplicato per **k** darà come risultato **1**.

Questo valore può essere utilizzato per eseguire una divisione tramite l'operazione di moltiplicazione, questo perché $x/k = x \cdot k^{-1}$.

Nell'aritmetica ordinaria l'inverso moltiplicativo di k equivale ad $1/k$, in quanto $k \cdot (1/k) = 1$.

Mentre all'interno di un modulo n l'inverso moltiplicativo k^{-1} viene definito come $k \cdot k^{-1} \equiv 1 \pmod{n}$, avendo quindi infinite soluzioni. Affinché k possa avere un inverso modulo n è necessario che il minimo comune multiplo tra k ed n sia 1 .

Proponendo come esempio dover risolvere l'operazione $4 / 2 \pmod{5}$, è necessario trovare innanzitutto l'inverso moltiplicativo di $2 \pmod{5}$, il quale è 3 perché $2 \cdot 3 \pmod{5} = 1$.

Successivamente è necessario eseguire la moltiplicazione $4 \cdot 3 \pmod{5}$ per avere come risultato la divisione $4/2$.

Queste proprietà dell'inverso moltiplicativo sono essenziali nella matematica modulare perché in essa solitamente vengono utilizzati solo numeri interi, non potendo quindi permettere l'uso di un'operazione come la divisione che causa o il problema di avere risultati decimali o di dover gestire il resto delle operazioni.

Elevazione esponenziale modulare ottimizzata

L'elevazione esponenziale può essere eseguita semplicemente calcolando l'esponente e applicando successivamente la riduzione modulo, come accade per la somma e la moltiplicazione. Tuttavia, specialmente quando si effettuano calcoli con numeri significativamente grandi, potrebbe essere necessario ottimizzare il processo per evitare di gestire numeri eccessivamente grandi. Ad esempio calcolare $3^{39} \pmod{13}$ senza ottimizzazioni comporterebbe la riduzione di un numero relativamente grande, aumentando la complessità del calcolo.

La tecnica più efficiente per eseguire calcoli esponenziali di questo tipo è l'**esponenziazione modulare per mezzo dei quadrati ripetuti**, la quale può essere riassunta con la seguente funzione ricorsiva:

$$a^x = \begin{cases} a \cdot (a^{x-1}), & \text{se } x \text{ è dispari,} \\ (a^{x/2})^2, & \text{se } x \text{ è pari.} \end{cases}$$

Applicando questa tecnica possiamo riscrivere il calcolo di $3^{39} \pmod{13}$ nel seguente modo:

$$3^{39} = 3 \cdot 3^{38} = 3 \cdot (3^{19})^2 = 3^2 \cdot (3^{18})^2 = 3^2 \cdot ((3^9)^2)^2 = 3^2 \cdot ((3 \cdot 3^8)^2)^2 = \dots = 3^2 \cdot ((3 \cdot ((3^2)^2)^2)^2)^2$$

Grazie a questa ottimizzazione, il calcolo diventa più semplice di quanto sembri: essenzialmente si riduce a due passaggi chiave, ovvero ' $3^2 \pmod{13} = 9$ ' e ' $9^2 \pmod{13} = 3$ '.

Quindi, un calcolo complesso come $3^{39} \pmod{13}$ viene semplificato a una serie di operazioni modulari gestibili e ripetuti, riducendo drasticamente la complessità computazionale.

Radice quadrata modulare

Il calcolo della radice quadrata di un valore a in un modulo n consiste nel trovare il valore x nella formula $x^2 \equiv a \pmod{n}$. Tuttavia, questa operazione non è sempre possibile, poiché non tutti i valori all'interno del modulo possiedono una radice quadrata. I numeri che ammettono una radice

quadrata vengono definiti **residui quadratici**, e circa la metà dei numeri in ogni modulo appartengono a questa categoria.

Prendendo come esempio il **modulo 7**, si procede a calcolare il quadrato di tutti i valori del modulo:

$$1^2 = 1 \equiv 1 \pmod{7}$$

$$2^2 = 4 \equiv 4 \pmod{7}$$

$$3^2 = 9 \equiv 2 \pmod{7}$$

$$4^2 = 16 \equiv 2 \pmod{7}$$

$$5^2 = 25 \equiv 4 \pmod{7}$$

$$6^2 = 36 \equiv 1 \pmod{7}$$

Questo significa che i **residui quadratici** nel modulo **7** sono **1, 2 e 4**, mentre **3, 5 e 6** non hanno una radice quadrata modulo 7.

L'algoritmo più noto per calcolare la radice quadrata di un valore all'interno di un modulo è **Tonelli-Shanks**, che funziona solo se il modulo è definito da un valore primo.

Se il modulo **n** non è primo **Tonelli-Shanks** non può esser applicato direttamente, ma è necessario fattorizzare **n** in valori primi **$p_1 * p_2 * \dots * p_k$** .

Una volta ottenuta la fattorizzazione, si devono calcolare le radici quadrate modulo **$p_1, p_2, \dots, p_k$** , e successivamente combinare i risultati utilizzando il **Teorema del resto cinese** per trovare la soluzione **x** che soddisfi simultaneamente le congruenze:

$$x \equiv x_1 \pmod{p_1} \quad x \equiv x_2 \pmod{p_2} \quad \dots \quad x \equiv x_k \pmod{p_k}$$

2.2 Problema del logaritmo discreto (teoria generale)

La sostanziale differenza tra un più noto **algoritmo continuo** e un **algoritmo discreto** sta nel contesto in cui essi vengono applicati.

Mentre il **logaritmo continuo** viene applicato su numeri reali per analisi matematiche e per risoluzione di problemi chiusi, definiti dalla formula $q = g^x$ (la cui soluzione è $x = \log_g q$), il **logaritmo discreto** viene utilizzato principalmente in contesti della teoria dei numeri e nella crittografia.

Generalmente un **logaritmo discreto** viene applicato su numeri interi definiti su un **campo finito** per risolvere congruenze nei problemi modulari espressi con la formula generica $q \equiv g \# x \pmod{p}$ dove $\#$ indica un operatore binario appartenente al **gruppo abeliano** (come ad esempio addizione, divisione ed elevazione esponenziale). Le quali sono definite dalle proprietà di **commutatività**, **associatività**, **esistenza dell'elemento neutro** ed **esistenza degli inversi**.

La formula $q \equiv g \# x \pmod{p}$ definisce la struttura matematica sulla quale il logaritmo discreto viene applicato. L'operatore involto nella formula varia a seconda del contesto applicato, ma persiste la struttura in cui q è congruente al risultato dell'operazione svolta tra g e x .

Di seguito vengono riportati tre esempi, due dei quali verranno approfonditi nelle successive sezioni:

-problema del logaritmo discreto su gruppi additivi: $q \equiv g * x \pmod{p}$

-problema del logaritmo discreto su gruppi moltiplicativi: $q \equiv g^x \pmod{p}$

-problema del logaritmo discreto su gruppi di punti di una curva ellittica: $q \equiv G \cdot x \pmod{p}$

La risoluzione dei problemi relativi agli algoritmi discreti è difficile e computazionalmente onerosa perché consiste nel risolvere una **One-Way Function** avente una moltitudine di possibili soluzioni. Il fatto che tutti i logaritmi discreti siano **One-Way Function** significa che la formula $q \equiv g \# x \pmod{p}$ è facile da calcolare, ma computazionalmente proibitivo da invertire a causa della complessità delle operazioni richieste, le quali hanno un tipo di complessità differente a seconda della struttura coinvolta.

Inoltre per tutti i tipi di logaritmi discreti esiste la **proprietà di omomorfismo**:

se $g \# x \equiv h$ e $g \# y \equiv k$ allora $(g \# x) \# (g \# y) \equiv h \# k$

Problema del logaritmo discreto su gruppi moltiplicativi

La struttura matematica del gruppo moltiplicativo in cui viene definito questo tipo di algoritmo discreto si compone di **operazioni moltiplicative ed esponenziali modulari**, applicando la riduzione modulare su ogni operazione eseguita.

Il **logaritmo discreto su gruppi moltiplicativi** è definito dalla risoluzione del problema dato dall'equazione $q \equiv g^x \pmod{p}$ che, a differenza del problema continuo $q = g^x$, non è risolvibile da una formula chiusa.

Questo perché i logaritmi hanno proprietà ben definite e possono essere calcolati con delle formule chiuse, ma quando si introduce la modularità nell'equazione esponenziale, trasformando l'equazione in " $q \equiv g^x \pmod{p}$ tale che esiste solo una ' x ' come soluzione", il problema cambia

drasticamente.

La riduzione modulare causa un'indeterminazione del risultato in quanto esso è "confinato" nell'intervallo $[0, p-1]$ a causa della riduzione modulare. In altre parole, molteplici valori di x possono produrre lo stesso valore q , come testimoniato dalla seguente definizione:

$q = g^x \pmod{p} = g^{x+k(p-1)} \pmod{p}$ per qualsiasi intero k .

Veloce dimostrazione:

$$\begin{aligned} & g^{x+k(p-1)} \pmod{p} && \text{riduzione esponente (} a^{x+y} = a^x * a^y \text{)} \\ & = g^x * g^{k(p-1)} \pmod{p} && \text{riduzione esponente (} a^{xy} = a^{yx} = (a^y)^x \text{)} \\ & = g^x * (g^{p-1})^k \pmod{p} && \text{teorema di Fermat (} a^{p-1} \pmod{p} \equiv 1 \text{)} \\ & = g^x * 1^k \pmod{p} = g^x \pmod{p} \\ & \Rightarrow g^x \pmod{p} = g^{x+k(p-1)} \pmod{p} \end{aligned}$$

In una situazione reale di violazione della chiave, questa proprietà rende estremamente difficile risolvere il problema perché non si vuole trovare uno dei possibili valori di x , ma bensì lo specifico valore di x che compone l'esponente $e = x+k(p-1)$, avendo come incognite sia x che k .

Questa situazione rende la procedura non un semplice calcolo risolvibile con una formula chiusa, ma una serie di operazioni, di speculazioni e di tentativi che possono facilmente diventare impraticabili con valori sufficientemente grandi.

Problema del logaritmo discreto su gruppi di punti di una curva ellittica

La struttura matematica del gruppo di punti in cui viene definito questo tipo di algoritmo discreto si compone di **operazioni di somma e raddoppio dei punti ellittici in un campo finito**, applicando la riduzione modulare su ogni operazione eseguita.

Il **logaritmo discreto su gruppi moltiplicativi** è definito dalla risoluzione del problema dato dall'equazione $Q \equiv G \cdot x \pmod{p}$, dove l'operatore “.” rappresenta il prodotto ECC tra un punto della curva ellittica ed un valore scalare (l'algoritmo ECC viene spiegato nel prossimo capitolo).

La difficoltà del **problema del logaritmo discreto nei gruppi di punti di una curva ellittica** sta nel fatto che la generazione delle chiavi su curve ellittiche non segue le operazioni di somma e moltiplicazione dei numeri interi, ma utilizza operazioni di **somma di punti sulla curva**, le quali non possono essere fattorizzate come avviene con i numeri interi su un campo finito.

Il calcolo inverso per ricavare il valore x partendo dalla formula $Q \equiv G \cdot x \pmod{p}$ non è rappresentabile da una formula analitica semplice. Questo perché il processo di calcolo della chiave pubblica si compone di una serie di calcoli aperti ridotti in modulo e non esiste un operatore inverso formale per la somma tra punti della curva ellittica.

Poiché la generazione della chiave pubblica Q richiede una serie di operazioni iterative, non esistono metodi noti efficienti per determinare la chiave privata x , se non mediante algoritmi che non differiscono molto dal metodo di un attacco di **brute force**.

Due dei metodi più conosciuti teoricamente utilizzabili per compromettere la chiave privata sono i seguenti elencati:

-**Baby-Step Giant-step (per gruppi moltiplicativi)**: Sfrutta la proprietà dei numeri per determinare un tetto massimo $b = \sqrt{n}$, memorizzare tutti i risultati di $G^j \mid j \in \{0, \dots, b-1\}$ per poi confrontarli con i risultati di $h * (g^{-b})^i \mid i \in \{0, \dots, b-1\}$ ed infine compararli tra loro per determinare il risultato.

-**Pollard's Rho**: Sfrutta la teoria dei cammini casuali per esaminare i campi finiti tramite collisioni casuali di calcoli iterativi.

I due algoritmi menzionati sono stati ideati per lavorare in un contesto di **gruppi moltiplicativi**, dunque richiedono un adattamento per poter funzionare in contesti di **gruppi di curve ellittiche**. Inoltre, in confronto al metodo di **brute force**, riducono la complessità da $O(n)$ a $O(\sqrt{n})$, il che è comunque troppo oneroso per gli attuali calcolatori in caso di chiavi di grandi dimensioni.

3 Algoritmi di firma digitale trattati

In questo capitolo viene spiegato il funzionamento degli algoritmi di firma digitale trattati, per poi effettuare delle analisi di confronto dal punto di vista di efficienza, complessità e sicurezza.

3.1 Digital Signature Algorithm (DSA)

Il **Digital Signature Algorithm (DSA)** è un sistema di criptaggio specializzato nella creazione di firme digitali che basa la sua sicurezza sulla difficoltà della reversibilità del **problema del logaritmo discreto su gruppi moltiplicativi** definito dal problema di trovare x nell'equazione $y = g^x \pmod{p}$.

Inizializzazione dei valori:

Per la generazione della firma tramite algoritmo **DSA** vengono estratti e derivati i valori che definiranno la **firma digitale**.

- Un numero primo casuale p che definisce il **campo finito**.
- Un numero primo casuale q tale che esso sia un **divisore primo** di $p-1$.
- Il valore g , definito come $g = h^{(p-1)/q} \pmod{p}$.
Il valore h è un numero casuale minore di p , inoltre g deve essere maggiore di 1.
- La chiave privata x , rappresentata da un numero casuale compreso nell'intervallo $[2, p-1]$.
- La chiave pubblica y , definita come $y = g^x \pmod{p}$.

Tra questi valori vengono resi pubblici y , q e g oltre alla **chiave pubblica** in modo da permettere ad altri utenti di verificare la **firma** che verrà generata.

L'esposizione di questi dati non causa significativi rischi sulla sicurezza con chiavi di dimensioni superiori ai **1024 bit**. Tale sicurezza è dovuta alla mera onerosità computazionale dei **calcoli logaritmici** necessari per ricavare la **chiave privata** pur conoscendo tutti i parametri utilizzati e la firma.

Generazione della firma:

La **firma digitale DSA** è composta dalla coppia di valori (r, s) , calcolati utilizzando un **valore temporaneo casuale** k , compreso nell'intervallo $[2, p-1]$, che viene estratto ad ogni nuova generazione di una firma:

- $r = (g^k \pmod{p}) \pmod{q}$
- $s = (k^{-1} * (H(m) + x*r)) \pmod{q}$ la variabile $H(m)$ rappresenta il valore in **bytecode del digest del file da firmare**.

Questa firma è sicura perché, volendo ricavare la chiave privata x , sarebbe necessario affrontare sia un **logaritmo discreto** al fine di dedurre k per poi risolvere l'**espressione lineare modulare** per ricavare x , rendendo la violazione della firma ancor più onerosa dell'approccio che sfrutta i valori pubblici.

Validazione della firma:

La verifica della **firma digitale DSA** consiste nel verificare la validità della **firma** dati i **valori pubblici** forniti e il **file firmato** ricevuto.

Questo controllo avviene eseguendo varie operazioni tra i **valori forniti** al fine di calcolare un valore **v** che dovrà risultare uguale al valore **r** della **firma** al fine di stabilirne la validità.

- $w = s^{-1} \pmod{q}$
- $u_1 = (H(m) * w) \pmod{q}$
- $u_2 = (r * w) \pmod{q}$
- $v = (g^{u_1} * y^{u_2} \pmod{p}) \pmod{q}$

3.2 Elliptic Curve Digital Signature Algorithm (ECDSA)

L'Elliptic Curve Digital Signature Algorithm (ECDSA) è una **variante del DSA** in cui la **chiave pubblica** viene generata tramite l'**algoritmo ECC**. Quindi la **chiave pubblica** non è rappresentata da un singolo valore numerico, ma da un **punto della curva ellittica**.

Le operazioni dell'algoritmo **ECDSA** per la generazione e la verifica della **firma** sono fortemente simili a quelle del **DSA**, avendo come unica differenza l'uso della somma tra punti dell'**ECC** anziché i calcoli con esponente.

Per la generazione della firma tramite algoritmo **ECDSA** vengono utilizzati i seguenti valori:

- Una curva ellittica ($y^2 = x^3 + ax + b$) tale che $4a^3 + 27b^2 \neq 0$
- Il numero primo p che definisce il **campo finito della curva ellittica**.
- Un punto generante casuale G appartenente alla **curva ellittica** definita all'interno del **campo finito**.

La **coordinata y** viene calcolata tramite la funzione della curva ellittica **modulo p** partendo da una **coordinata x** casuale tale che la **coordinata y** risultante sia un valore intero.

- Il valore q equivalente al grado del punto G .
- La chiave privata x , rappresentata da un numero casuale compreso nell'intervallo $[2, n-1]$.
- La chiave pubblica Y , definita come $Y = x \cdot G$.

Tra questi dati vengono resi pubblici G , q , p ed a oltre alla **chiave pubblica** in modo da permettere poi la verifica della firma.

Generazione della firma:

La firma digitale **ECDSA** è composta dalla coppia di valori (r, s) , calcolati utilizzando un **valore temporaneo casuale k**, compreso nell'intervallo $[2, n-1]$, che viene estratto ad ogni nuova generazione di una firma:

- $R = k \cdot G$ da cui si ricavano la coordinata x_R
- $r = x_R \pmod{q}$ con $r \neq 0$
- $s = (k^{-1} * (H(m) + x * r)) \pmod{q}$ con $r \neq 0$ e la variabile $H(m)$ rappresentante il valore in **bytecode del digest del file da firmare**.

Questa **firma** è particolarmente **sicura** perché vengono eseguite **operazioni su gruppi di punti su una curva ellittica**, ovvero strutture matematiche tra le più complesse tra quelle utilizzate attualmente.

Come per l'algoritmo **DSA**, la sicurezza della firma si basa sull'onerosità computazionale dei calcoli, i quali in questo caso sono relativi al **problema del logaritmo discreto su gruppi di punti su una curva ellittica**.

Validazione della firma:

La verifica della **firma digitale ECDSA** avviene quasi esattamente come nel logaritmo **DSA**; eseguendo operazioni atte a verificare la validità della **firma** dati i **valori pubblici** forniti e il **file firmato** ricevuto.

Le operazioni eseguite sono finalizzate al calcolo del punto **V** per poi comparare la coordinata **x_v** con l'elemento **r** della **firma (r, s)**.

Le operazioni che includono punti vengono risolte con le operazioni previste dall'algoritmo **ECC**.

- $w = s^{-1} \pmod{q}$
- $u_1 = (H(m) * w) \pmod{q}$
- $u_2 = (r * w) \pmod{q}$
- $V = (u_1 \cdot G) + (u_2 \cdot Y)$

3.2.1 Elliptic Curve Cryptography (ECC)

L'**Elliptic Curve Cryptography (ECC)** è un sistema crittografico utilizzato per **generare coppie di chiavi utilizzando operazioni di raddoppio e di somma tra punti su una curva ellittica** ($y^2 = x^3 + ax + b$).

Dati un **punto della curva G** e una **chiave privata d**, è possibile calcolare la **chiave pubblica Q** con l'operazione $Q = G_1 + G_2 \dots + G_d$.

La **somma tra punti** non si svolge sommando le coordinate dei due punti come in una consueta somma vettoriale, ma nella somma $R = P + T$ viene eseguita un'**operazione geometrica** consistente nel tracciare una retta passante per i due punti **P** e **T**, ricavare **R'** dall'intersezione della retta con la curva della funzione e in fine specchiare il punto **R'** secondo l'asse x per ricavare il punto **R**.

Nel caso i punti **P** e **T** siano lo stesso punto ($R = 2 * P$) la retta utilizzata è la tangente della curva.

Per replicare matematicamente queste **operazioni geometriche** esistono i seguenti due metodi:

- $R = P + P$: (anche definito come $R = 2P$)
 $s = (3 * x_P^2 + a) / (2 * y_P) \pmod{p}$
 $x_R = s^2 - 2 * x_P \pmod{p}$
 $y_R = s * (x_P - x_R) - y_P \pmod{p}$
- $R = P + T$:
 $s = (y_T - y_P) / (x_T - x_P) \pmod{p}$
 $x_R = s^2 - x_P - x_T \pmod{p}$
 $y_R = s * (x_P - x_R) - y_P \pmod{p}$

3.2.2 Ordine di un punto su una curva ellittica

L'**ordine di un punto P** nel contesto di una **curva ellittica** equivale al valore minimo **n** affinché la moltiplicazione scalare $P \cdot n$ dia come risultato un **punto all'infinito**.

Un **punto all'infinito** (indicato col simbolo **O**) è un **punto ideale non rappresentabile** sul piano cartesiano che viene utilizzato per rappresentare la direzione comune di tutte le rette parallele in geometria proiettiva.

In un contesto di somma di punti su una curva ellittica, il punto all'infinito funge da **elemento neutro** dell'operazione di somma, cioè un punto tale che, quando sommato a un altro punto della curva, restituisce lo stesso punto.

Per calcolare il punto all'infinito è necessario eseguire il raddoppio del punto iniziale **P** per poi continuare ad incrementare il risultato addizionandolo con **P** finché il risultato non diventa il **valore neutro della somma**, portando quindi ad una situazione in cui $P + K = P$.

$$2P = K_2 \rightarrow K_2 + P = K_3 \rightarrow \dots \rightarrow K_n + P = P$$

$$K_n + P = P \rightarrow K_n = O \rightarrow \text{grado di } P = n$$

Coordinate proiettive

In applicazioni di crittografia reali, per **calcolare e rappresentare il punto all'infinito** in maniera funzionale non è sufficiente la consueta rappresentazione cartesiana con due coordinate.

Lavorando al progetto ho riscontrato una cruciale discordanza con la teoria. Di fatto non mi risultava possibile, adoperando ricorrenti somme tra punti a due coordinate (anche detti **punti affini**), ricavare un punto **O** tale che $O + P = P$.

Dunque, in seguito ad ulteriori ricerche, ho scoperto l'esistenza di altri sistemi di coordinate, tra cui le **coordinate proiettive**.

Queste coordinate sono composte dalle coordinate (x, y, z) , dove la coordinata **z** non rappresenta la terza dimensione nello spazio, ma estende le coordinate affini (x, y) per provvedere ad una rappresentazione alternativa di questi punti.

Tale rappresentazione è specificatamente atta alla semplificazione di operazioni relative al punto all'infinito.

Essendo le coordinate proiettive una diversa rappresentazione di quelle affini, i punti proiettivi hanno dei corrispettivi punti affini e viceversa, più precisamente **un punto affine coincide con uno o più punti proiettivi**.

Per questo motivo è possibile **convertire un punto proiettivo (x, y, z) in un punto affine $(x/z, y/z)$** e **convertire un punto affine (x, y) in un punto proiettivo $(x, y, 1)$** .

Il punto all'infinito è rappresentato con la coordinata $z = 0$, in questo modo **le sue coordinate proiettive $(x/z, y/z)$ risultano avere un valore infinito**.

Raddoppio e somma dei punti

L'aggiunta della coordinata **z** sconvolge il calcolo del raddoppio e della somma dei punti, portando le operazioni a diventare una serie di calcoli con numerosi valori appendice.

Non sono riuscito a trovare definizioni matematiche sulle operazioni necessarie, quindi di seguito riporto i due **pseudo-codici** che sono riuscito a ricavare analizzando le **implementazioni del codice C** ottenute dalla **libreria CycloneCrypto della Oryx Embedded** (ampiamente trattata nell'ultimo capitolo):

Calcolo di 2P:

```
A = zR^4 * a
B = xR^2
A = A + (3 * B)
zR = zR * yR * 2
yR = yR^2
B = xR * yR * 4
xR = A^2 - (2 * B)
yR = yR^2 * 8
B = (B - xR) * A
yR = B - yR
```

Calcolo di P+Q:

```
A = xQ
B = yQ
C = zQ
V = C^2
xR = xR * V
V = V * C
yR = yR * V
V = zR^2
A = A * V
V = zR * V
B = B * V
A = xR - A
B = yR - B
if (A == 0) {return pointToInfinity}
xR = (2 * XR) - A
yR = 2 * yR - B
zR = zR * C * A
V = A^2
A = A * V
V = V * xR
xR = B^2
xR = xR - V
V = V - (2 * xR)
B = B * V
A = A * yR
yR = B - A
yR = yR / 2
```


3.3 DSA ed ECDSA a confronto

Dimensione delle chiavi:

Sia l'**Elliptic Curve Digital Signature Algorithm (ECDSA)** che la **Digital Signature Algorithm (DSA)** sono utilizzati per creare **firme digitali** sicure, ma la loro sicurezza si basa su **principi matematici** differenti.

Le **operazioni logaritmiche** necessarie per decifrare in maniera illecita le informazioni criptate con **DSA** incrementano di difficoltà in maniera **sub-esponenziale** rispetto alla dimensione della chiave. Mentre, nel caso dell'**ECDSA**, la difficoltà delle **operazioni logaritmiche** cresce **esponenzialmente** rispetto alla dimensione della chiave.

Questo significa che, per ottenere lo stesso livello di sicurezza, **ECDSA** richiede chiavi molto più piccole rispetto a **DSA**.

Ad esempio, una chiave **ECDSA** di **256 bit** offre un livello di sicurezza equivalente a una chiave **DSA** di **3072 bit**.

Differenze strutturali:

Essendo **ECDSA** una **variante di DSA**, i due algoritmi hanno una struttura molto simile per il **calcolo delle chiavi**, l'attribuzione della **firma** e la verifica della **firma**.

La principale differenza tra i due algoritmi, oltre alla struttura matematica utilizzata, risiede nella generazione del valore **g** (definito **G** nell'**ECDSA**), nella rappresentazione delle **chiavi** e nel processo di scelta dei moduli.

Nel **DSA** vengono scelti un numero primo casuale **p**, un numero primo **n** che divide **p-1** ed un numero casuale **h**, per poi calcolare il valore $g = h^{(p-1)/n} \bmod p$.

Mentre nell'**ECDSA** viene calcolato un punto base **G** appartenente alla curva ellittica, per poi calcolare il **modulo n** tale che **n = ordine del punto G**.

Il fatto che l'**ECDSA** utilizzi un punto anziché un valore scalare come valore **g** causa una variazione delle formule che includono **g** (o **G**).

Laddove nel **DSA** vengono eseguite delle **operazioni di potenza modulare**, nell'**ECDSA** vengono eseguiti **calcoli di complessità lineare**.

Questa alterazione porta nell'**ECDSA** a delle operazioni meno onerose da calcolare, anche se questa differenza è meno evidente con l'utilizzo di **parametri precalcolati**.

Inoltre i parametri privati **ECDSA** sono più onerosi da violare in quanto la struttura matematica di **gruppi di punti su una curva ellittica** è più complessa ed imprevedibile di quella di un **gruppo moltiplicativo**.

4 DSA ed ECDSA: implementazione degli algoritmi

In questa sezione finale, viene presentata la documentazione delle implementazioni sviluppate per dimostrare il funzionamento degli algoritmi di firma digitale **DSA** ed **ECDSA**.

Il **progetto** è stato creato a scopo dimostrativo e include quattro programmi eseguibili, situati nelle due cartelle **C/src/examples** e **Java/src/examples**.

Ogni programma consiste in un **main** che genera localmente tutti i **valori necessari**, applica una **firma** digitale e, infine, verifica la validità della **firma**.

Al fine di facilitare la comprensione delle operazioni eseguite, gli esempi inclusi nei **main** sono stati progettati per stampare in console tutti i valori in chiaro.

Inoltre, solamente nelle due implementazioni **C**, i valori iniziali forniti (come la **chiave privata**, i **moduli** e le **variabili della curva ellittica**) sono nell'ordine di grandezza di 10^2 , per consentire un confronto manuale con le operazioni teoriche.

Il repository **GitHub** del **progetto**, a cui questo documento fa riferimento, è disponibile al seguente link:

<https://github.com/Marclova/CryptographyImplementations.git>

4.1 Oracle (JAVA)

Per le due implementazioni in **Java** mi sono servito delle librerie della **Oracle**, le quali, pur essendo eccessivamente sintetiche e difficili da navigare, sono complete, affidabili e veloci da implementare.

Essendo quelle della **Oracle** delle librerie nativamente inserite all'interno di **Java**, le ho ritenute una soluzione semplice ed autorevole, permettendomi di utilizzarle senza dover appesantire il progetto aggiungendo librerie o classi esterne.

Per realizzare il progetto è stato utilizzato il pacchetto **Java.Security**, il quale compone la libreria **Java Cryptography Extensions (JCE)**.

Quest'ampia libreria estende il framework **Java Cryptography Architecture (JCA)**, fornendo funzionalità avanzate di crittografia che includono algoritmi di crittografia a chiave simmetrica e asimmetrica, generatori di numeri casuali e algoritmi di firma.

4.1.1 Java Security

Molte delle **classi** definite nella **Java.Security** sono delle **classi astratte** che fanno uso dello **Factory Method**, applicato tramite l'utilizzo del metodo **getInstance(String algorithm)**.

Il **nome** delle implementazioni dell'algoritmo inserito come parametro nel metodo **getInstance** deve essere incluso tra varie opzioni presenti nella lista dei **Java Security Standard Algorithm Names** della **Java.Security**.

I metodi che eseguono le funzionalità delle **classi** definite nella **Java.Security** sono vuote perché

sono pensate per essere sostituite da dei **sottotipi implementati** in runtime, tramite il metodo **getInstance**.

Ad esempio, la classe astratta **KeyPairGenerator** ha come responsabilità quella di generare una coppia di chiavi, ma un oggetto di questa classe deve essere istanziato tramite il metodo **getInstance**, specificando come parametro il nome dell'algoritmo da utilizzare per la generazione delle chiavi.

In pratica, viene creato un oggetto che implementa l'interfaccia **KeyPairGenerator**, con le implementazioni di una **classe concreta** selezionata da un catalogo di classi disponibili.

La **Java.Security** è un pacchetto molto agile da implementare all'interno di grandi progetti perché le sue classi sono progettate in modo modulare e flessibile grazie all'uso di interfacce e del **Factory Method**.

Questo approccio permette alle classi di supportare diverse implementazioni degli algoritmi crittografici, che possono essere facilmente intercambiate anche in runtime.

In questo modo, è possibile adattare il comportamento dell'applicazione a diversi requisiti di sicurezza senza modificare il codice che utilizza le interfacce crittografiche, migliorando l'estensibilità e la manutenibilità del software.

Link per la documentazione ufficiale della libreria:

<https://docs.oracle.com/javase%2F8%2Fdocs%2Fapi%2F%2F/java/security/package-summary.html>

Link per la lista dei Java Security Standard Algorithm Names:

<https://docs.oracle.com/en/java/javase/11/docs/specs/security/standard-names.html>

Personalizzazione dei parametri

La **Java.Security** di per sé non dispone di classi capaci di calcolare nessuno dei valori utilizzati per l'applicazione della firma, ma prevede l'utilizzo di parametri standard come la curva ellittica **secp256r1**.

Quindi è possibile generare delle firme, ma senza la possibilità di scegliere né i parametri né la chiave privata utilizzata. Il che non permette di sfruttare due importanti caratteristiche della firma digitale; l'**autenticazione** e il **non-ripudio**.

In una firma digitale la chiave privata è un componente fondamentale che identifica l'identità del firmatario. Utilizzare una chiave privata casuale generata dalla classe

Java.Security.KeyPairGenerator equivale a firmare un documento cartaceo utilizzando un nome a caso, quindi è stato necessario implementare la classe astratta **AlgorithmParametersCalculator** e i rispettivi sottotipi per permettere l'uso di chiavi private inserite manualmente.

Metodo getInstance(...)

La presenza del **metodo statico getInstance** indica l'uso del **Factory Method Pattern**. Questo **design pattern** permette di centralizzare la logica di creazione degli oggetti attraverso un'**interfaccia** o una **classe astratta**, delegando la scelta delle implementazioni specifiche alle classi

concrete.

In altre parole, il **Factory Method** consente di definire operazioni basate su un tipo astratto, lasciando che, al momento dell'esecuzione, venga restituita l'implementazione concreta adatta tramite `getInstance`.

Nel contesto della libreria **Java.Security**, ogni **classe astratta**, come ad esempio **KeyPairGenerator** o **Signature**, definisce un metodo **getInstance** che in almeno uno dei suoi **overload** richiede il **nome dell'algoritmo** in formato stringa (es. "DSA" per la generazione di chiavi o "SHA256withDSA" per la cifratura).

In aggiunta, è possibile specificare un **provider**, che rappresenta l'implementazione concreta dell'algoritmo, ovvero un insieme di classi che forniscono l'algoritmo richiesto e i relativi calcoli crittografici.

In assenza di **parametri** espliciti per l'algoritmo, il **provider** o il **generatore di numeri casuali**, l'istanza restituita da **getInstance** utilizzerà **valori standard** definiti per quell'algoritmo. Questi valori predefiniti solitamente garantiscono che l'oggetto creato sia immediatamente utilizzabile con le configurazioni più comuni.

Classe BigInteger

Data la necessità di utilizzare numeri notevolmente grandi non è possibile utilizzare variabili primitive e per questo motivo viene utilizzata questa classe.

Gli oggetti di questa classe contengono i valori in **formato binario**, immagazzinando i dati come **array di bytes**, permettendo di immagazzinare valori compresi nell'intervallo $[-2^{\text{Integer.MAX_VALUE}}, 2^{\text{Integer.MAX_VALUE}}]$.

La classe **BigInteger** implementa tutte le operazioni basilari (lineari e in modulo) tra **grandi valori binari**. Queste operazioni possono essere facilmente estese per supportare calcoli più complessi. Inoltre è possibile concatenare varie operazioni in modo da eseguire intere espressioni con un solo assegnamento. Di seguito viene mostrato un esempio:

```
// compute g = ( h^((p-1)/n) )
return h.modPow(
    field.subtract(BigInteger.ONE)
        .multiply(q.modInverse(field)).mod(field),
    field
);
```

Interfacce di marcatura AlgorithmParameterSpec

All'interno della **Java.Security** vengono utilizzate delle interfacce di marcatura per categorizzare le varie classi che vengono implementate nella libreria.

Queste interfacce non garantiscono alcuna proprietà comune tra le classi che le implementano, ma solo uno scopo comune.

La libreria **Java.Security** è strutturata con delle **classi astratte** che accettano come tipo dei

parametri le interfacce di marcatura, ma definendo delle classi concrete che accettano solo classi specifiche.

AlgorithmParameterSpec

L'interfaccia **AlgorithmParameterSpec** rappresenta la specifica dei parametri utilizzati da un determinato algoritmo di firma digitale (ex. "ECParameterSpec").

Le classi della **Java.Security** che implementano **AlgorithmParameterSpec** possono esser create da un costruttore che richiede di inserire tutti i parametri già calcolati e non permettono di modificare i parametri delle istanze. Quindi per aggiornare un oggetto di tipo **AlgorithmParameterSpec** è necessario creare una nuova istanza.

PublicKey e PrivateKey

Le interfacce **PublicKey** e **PrivateKey**, che implementano l'interfaccia **Key**, rappresentano delle **chiavi generiche** che verranno utilizzate come **sovratipo** delle effettive chiavi utilizzate (come ad esempio **DsaPublicKey** per un algoritmo **DSA**), permettendo l'utilizzo di qualunque tipo di chiave. Le **classi** che implementano queste interfacce non forniscono i propri parametri in chiaro, rendendole un'opzione sicura ed ampiamente utilizzata dai metodi della **Java.Security**.

PublicKeySpec e PrivateKeySpec

Le interfacce **PublicKeySpec** e **PrivateKeySpec**, che implementano l'interfaccia **KeySpec**, rappresentano delle **specifiche di chiavi generiche** ovvero l'insieme di parametri e attributi che compongono le chiavi rappresentate (come ad esempio **DsaPublicKeySpec** per un algoritmo **DSA**). Le **classi** che implementano queste interfacce forniscono i propri parametri in chiaro, rendendole una valida opzione per implementazioni basate sulla manipolazione dei dati.

Classe astratta KeyPairGenerator

Questa **classe astratta** viene utilizzata per generare una coppia di chiavi, ovvero un oggetto **KeyPair**, contenente la coppia di chiavi pubblica e privata, sfruttando una delle implementazioni definite nei **Java Security Standard Algorithm Names** della libreria **Java.Security**.

L'istanza di **KeyPairGenerator** viene ottenuta tramite il metodo statico **getInstance**, che, grazie ai suoi **overload**, può accettare anche un **provider** o un **generatore di numeri casuali**.

Una volta creata, l'istanza può essere inizializzata tramite il metodo **initialize**, che dispone di quattro **overload**. Questi consentono di specificare vari parametri come la **dimensione delle chiavi** o la classe **SecureRandom** per la generazione di **numeri casuali** crittograficamente sicuri.

Tuttavia, poiché le istanze di **KeyPairGenerator** hanno **parametri e un generatore di numeri casuali predefiniti**, l'utilizzo del metodo **initialize** è **facoltativo**, soprattutto se si utilizza il generatore in una soluzione che adopera **valori standard** conformi al **FIPS**.

Il metodo principale che effettivamente adempie alla funzionalità della classe è **generateKeyPair()**, il quale genera e ritorna un **KeyPair** contenente la **coppia di chiavi pubblica e privata**.

Di seguito è riportato un esempio d'uso di **KeyPairGenerator** per generare due chiavi per l'algoritmo **ECDSA**:

```
KeyPairGenerator kPairGen = KeyPairGenerator.getInstance("EC");
KeyPair keyPair = kPairGen.generateKeyPair();
PublicKey publicKey = KeyPair.getPublic();
PrivateKey privateKey = KeyPair.getPrivate();
```

Classe KeyFactory

Questa **classe** viene utilizzata per convertire una **PublicKeySpec** o una **PrivateKeySpec** nelle rispettive **PublicKey** o **PrivateKey** e viceversa.

Nonostante **KeyFactory** sia una classe concreta, essa utilizza il metodo statico **getInstance(String algorithm)**.

I due metodi **generatePrivate(KeySpec keySpec)** e **generatePublic(KeySpec keySpec)** generano rispettivamente una **PrivateKey** o una **PublicKey** basata sulla **KeySpec** fornita come parametro. Questi metodi accettano un parametro di tipo generico **KeySpec**, che può essere, ad esempio, una **ECPrivateKeySpec** o una **DSAPublicKeySpec**.

Tuttavia, se il **KeySpec** passato non è compatibile con il tipo di chiave richiesto dal metodo (**generatePrivate** o **generatePublic**), viene lanciata un'eccezione **InvalidKeySpecException**. Questo accade perché, pur accettando un parametro di tipo generico, ogni metodo richiede uno specifico sottotipo di **KeySpec**.

Il metodo **getKeySpec(Key key, Class<T> keySpec)** rappresenta l'operazione inversa dei due metodi "generate", convertendo un **sottotipo di Key** in lo **specifico sottotipo di KeySpec**, definito dal tipo **T** rappresentato dall'oggetto **Class<T>**.

Anche questo metodo lancia un **InvalidKeySpecException** se i due sottotipo di **Key** e **KeySpec** non sono compatibili.

```
KeyFactory kFactory = KeyFactory.getInstance("DSA");
PublicKey publicKey = kFactory.generatePublic(dsaPublicKeySpec);
DSAPrivateKeySpec privateKeySpec = kFactory.getKeySpec(privateKey,
                                                         DSAPrivateKeySpec.class);
```

Classe astratta Signature

Questa **classe astratta** viene utilizzata per creare e verificare le firme utilizzando qualunque algoritmo definito tra i **Security Standard Algorithm Names** della **Java.Security** tramite l'uso di **getInstance**. L'**algoritmo** ed il **provider** inseriti devono essere gli stessi utilizzati per generare le chiavi al fine di generare una **firma valida**.

Una volta creata l'**istanza di Signature**, essa deve essere inizializzata con uno dei due metodi **initSign(PrivateKey privateKey)** o **initVerify(PublicKey publicKey)**.

Dopo l'inizializzazione è necessario caricare il file coinvolto nella firma in formato **byte array** all'interno dell'istanza tramite il metodo **update**, il quale, tramite i suoi **overload**, permette di

concatenare all'attuale **byte array** (inizialmente vuoto) singoli **byte**, un **byte array**, una partizione di **byte array** o un **buffer** contenente il file in formato **byte array**.

Successivamente è possibile procedere all'**applicazione** o alla **verifica** della **firma** rispettivamente tramite i metodi **sign()** e **verify()**.

Verrà lanciato un **SignatureException** se si tenterà di eseguire una di queste due operazioni senza aver inizializzato l'istanza o dopo averla inizializzata per eseguire l'altra operazione.

Di seguito è riportato un esempio d'uso di **Signature** per generare e verificare una firma:

```
Signature sign = Signature.getInstance("DSA");
sign.initSign(privateKey);
sign.update(fileToSignAsByteArray);
byte[] generatedSignature = sign.sign();

sign.initVerify(publicKey);
boolean signatureIsValid = sign.verify(generatedSignature);
```

4.1.2 NativeDS

La libreria **NativeDS (Native Digital Signature)** è stata da me implementata per poter permettere al progetto di supportare operazioni di **calcolo nativo dei parametri**, gestione di **chiavi con valori scelti** dall'utente e **agevolazioni** per l'implementazione e l'impostazione degli algoritmi della **Java.Security**.

Quindi le classi e le implementazioni presenti in questa libreria sono strettamente dipendenti dalla **Java.Security** in quanto essa utilizza tutte le classi menzionate nel precedente paragrafo.

La libreria si divide in due **pacchetti**, ovvero **parameters** e **util**, responsabili rispettivamente della gestione dei dati e del provvedimento di quello che mi piace definire **zucchero sintattico**, ovvero l'insieme di **alias** ed **implementazioni** che riducono la verbosità del codice per renderlo più agile e leggibile.

Classe astratta ParametersCalculator

La classe astratta **ParametersCalculator**, definita nel pacchetto **NativeDSa.parameters**, fornisce le operazioni necessarie per il calcolo dei parametri di un **algoritmo di firma digitale generico**.

Le **classi concrete DSAParametersCalculator** e **ECParametersCalculator** estendono questa **classe astratta**, implementando i calcoli specifici per i rispettivi algoritmi.

Ispirandomi alle classi astratte della **Java.Security**, ho aggiunto alla classe una versione semplificata di **getInstance** per migliorare la portabilità delle sue sottoclassi.

Il **metodo statico** in questione è **simpleGetInstance(String algorithm)**, che restituisce una delle due sottoclassi: **DSAParametersCalculator** (restituito se si passa "DSA") oppure **ECParametersCalculator** (restituito se si passa "EC"). Entrambe le classi si trovano all'interno del pacchetto **parameters**, insieme a questa classe.

Questa **classe astratta** definisce tre metodi per il **calcolo nativo** dei parametri:

-Il metodo **calculatePrivateKeySpec(byte[] privateKeyValue, AlgorithmParameterSpec params)** serve unicamente per una conversione dei dati grezzi della chiave privata in una sottoclasse di **KeySpec** al fine di permettere una compatibilità con le classi della **Java.Security**.

-Il metodo **calculatePublicKeySpec(byte[] privateKeyValue, AlgorithmParameterSpec params)** implementa il **calcolo nativo della chiave pubblica**, ritornando una sottoclasse di **KeySpec** come risultato.

-Il metodo **calculateGValueAndUpdateParameterSpec(AlgorithmParameterSpec params, SecureRandomGenerator srg)** è leggermente controverso, poiché, dopo aver calcolato il valore del **generatore g**, invece di restituirlo direttamente, crea un **nuovo set di parametri aggiornati** rappresentato dallo stesso sottotipo di **AlgorithmParameterSpec** ricevuto.

Questa scelta è dovuta al fatto che l'interfaccia **AlgorithmParameterSpec** della **Java.Security** non prevede la modifica delle istanze, rendendo quindi necessaria l'inizializzazione di una nuova istanza di classe.

Di seguito viene riportato come utilizzare un'istanza di **ParametersCalculator** per calcolare il generatore **g** e la chiave pubblica di un **DSAParameterSpec** dati **p**, **q** e il valore grezzo della chiave privata:

```
DSAParametersCalculator dsaCalculator = ParametersCalculator
    .simpleGetInstance("DSA");
DSAParameterSpec parameters = new DSAParameterSpec(pValue, qValue, null);
parameters = dsaCalculator.calculateGValueAndUpdateParamSpec(parameters);
```

Classe astratta ParametersExtractor

La classe astratta **ParametersExtractor**, definita nel pacchetto **NativeDS.parameters**, definisce i metodi per poter estrarre informazioni dalle chiavi e dai parametri, i quali vengono implementati dalle due sottoclassi **ECPParametersExtractor** e **DSAPParametersExtractor**.

Ho deciso di creare un'intera gerarchia di classi per l'estrazione di informazioni da istanze di altre classi perché tali informazioni non sono facilmente accessibili, e la procedura di accesso varia notevolmente tra chiavi e parametri di algoritmi diversi, anche se implementano le stesse interfacce.

Ad esempio, **ECPublicKeySpec** e **DSAPublicKeySpec** possiedono strutture dati molto diverse; mentre la chiave **DSA** fornisce direttamente il proprio valore **BigInteger y**, la chiave **EC** contiene un'istanza **ECPParameterSpec**, la quale, a sua volta, contiene un'istanza di **EllipticCurve**, dove è possibile accedere al valore **ECPoint y** (denominato **W**).

Le sottoclassi hanno solo un costruttore vuoto, e i loro parametri non sono inizializzati. Questo design è stato scelto perché la classe è stata concepita per aggiornarsi attraverso i propri metodi di estrazione.

Quando un'istanza che estende **ParametersExtractor** esegue un'estrazione tramite uno dei metodi "**extractFrom**", aggiorna i propri parametri con quelli appena estratti.

Di seguito viene riportato un esempio di come **ECPParametersExtractor** viene utilizzato per semplificare l'estrazione di valori da un'istanza di **ECPParameterSpec**:

```
ECPParametersExtractor ecExtractor = new ECPParametersExtractor();
ecExtractor.extractFromParameterSpec(ecParamSpec);
BigInteger a = ecExtractor.getA();
BigInteger b = ecExtractor.getB();
BigInteger field = ecExtractor.getP();
BigInteger order = ecExtractor.getQ();
ECPoint gPoint = (ECPoint) ecExtractor.getG();
```

Classe OperationsManager

La classe **OperationsManager**, definita nel pacchetto **NativeDS.util**, viene utilizzata per semplificare l'implementazione del calcolo nativo di parametri personalizzati all'interno del codice.

Questa classe centralizza la logica delle istanziazioni e delle chiamate ai metodi principali, migliorando la modularità e la manutenibilità del codice.

Ognuno dei quattro metodi di **OperationsManager** svolge interamente una delle quattro operazioni principali della firma elettronica, vale a dirsi **calcolo del generatore**, **calcolo della coppia di chiavi**, **applicazione della firma** e **verifica della firma**, utilizzando le implementazioni fornite da **NativeDS** e da **Java.Security**.

Il costruttore richiede i nomi dell'**algoritmo di generazione delle chiavi** e dell'**algoritmo di hashing**, selezionabili tra quelli definiti tra i **Java Security Standard Algorithm Names**.

In modo opzionale, è possibile aggiungere un'istanza di **SecureRandom** per inizializzare l'oggetto **SecureRandomNumberGenerator**, che altrimenti verrà inizializzato tramite il suo costruttore vuoto.

Classe SecureRandomNumberGenerator

La classe **OperationsManager**, definita nel pacchetto **NativeDS.util**, implementa dei metodi che usufruiscono della classe **SecureRandom** della **Java.Security** per gestire numeri molto grandi tramite l'ausilio di **byte arrays** e valori **BigInteger**.

Questa classe genera numeri casuali compresi in un determinato intervallo definito da valori **BigInteger** o dal numero di byte specificato.

Il principio su cui si basa la generazione di numeri pseudo-casuali definiti all'interno di determinati intervalli sta nello stabilire di quanti byte il numero deve essere composto.

Dopo la generazione casuale dei byte, vengono applicate operazioni modulari per garantire che il valore rientri all'interno degli intervalli definiti e garantendo che il numero sia positivo tramite il controllo del bit del segno.

4.1.3 Implementazioni esterne

Per poter eseguire in Java il calcolo della radice quadrata modulare ho dovuto aggiungere al progetto un file trovato su **GitHub**.

Il file in questione è **TonelliShanks.java**, ovvero una classe stand-alone trovata nella repository raggiungibile tramite il seguente link:

<https://gist.github.com/ashu-tosh-kumar/7f0ed35befaf580875ff9ed5ad706cd6>

Questa classe presenta un solo metodo statico che richiede come parametri il valore **n** da cui estrarre la radice e il modulo **p**, per poi ritornare il risultato, se esiste.

Il metodo presenta una computazione più immediata nel caso specifico in cui il modulo **p** $\equiv 3 \pmod{4}$, ma non presenta una gestione della casistica in cui **p** non sia primo.

Infatti questa classe non prevede di eseguire il calcolo con moduli composti, quindi per utilizzarla in quella situazione è necessario implementare la fattorizzazione del modulo e l'applicazione del **teorema cinese del resto** sui risultati delle fattorizzazioni del modulo non-primo **p**.

4.1.4 Configurazioni per DSA ed ECDSA in Java

Il modo in cui la **JCE** è progettata, nello specifico esaminando la **Java.Security**, permette di sfruttare il **polimorfismo** delle classi **Java** tramite la programmazione astratta, basata sull'utilizzo di strutture basate sull'utilizzo di classi astratte ed interfacce.

Nel progetto allegato a questo documento vengono presentati due esempi di implementazione che, tramite uno switch inserito per motivi dimostrativi, permettono sia di effettuare una firma in DSA che in ECDSA con il medesimo codice sfruttando i tipi interfaccia e qualche dovuto controllo sul tipo eseguito in runtime tramite **instanceOf**.

Purtroppo per l'algoritmo **ECDSA** non sono riuscito ad implementare il calcolo dell'ordine di un punto sulla curva ellittica perché non sono riuscito a fornire le dovute ottimizzazioni del codice, portando ad un'esecuzione che non può essere svolta in tempi ragionevoli (ho stimato un'attesa di 3 anni per calcolare un modulo da 256 bit).

Inoltre sono stato anche limitato dalla **Java.Security** in quanto non risulta supportare le classi **ECParameterSpec**, **ECPrivateKeyImpl** ed **ECPublicKeyImpl**.

Però, non volendo optare per un'abolizione dell'uso di questa libreria **Oracle**, ho individuato nella libreria **openjdk-jdk11**, fornita tramite **GitHub** dalla **AdoptOpenJDK**, una possibile soluzione. Infatti nel pacchetto **jdk.crypto.ec** di tale libreria vengono definite le classi necessarie per la definizione delle chiavi **ECDSA**. (link del repository GitHub presente nella bibliografia)

A causa del tempo stringente, però, non sono riuscito ad implementare questa partizione di libreria all'interno del progetto.

L'inclusione della nuova libreria non è stata resa possibile nel breve periodo a causa di incompatibilità con la struttura del progetto, più precisamente problematiche causate dall'incompatibilità della classe **ECKeyFactory** con **KeyFactory**, che richiede un **refactoring di OperationManager** ed **ECParameterCalculator** o la **realizzazione di un bridge design pattern** per essere risolto.

Per quanto riguarda l'implementazione di **DSA** ed **ECDSA** con valori standard per-calcolati invece, sono riuscito con successo a creare un main che permette di intercambiare tra le due firme semplicemente cambiando il valore delle due stringhe utilizzate come parametri per i **getInstance**.

Configurazioni di avvio

Per eseguire la firma elettronica **DSA** o **ECDSA** in un'applicazione Java con la libreria **NativeDS**, è consigliato l'uso della classe **OperationsManager**.

Come spiegato nel paragrafo apposito, la classe **OperationsManager** è stata progettata sia per integrare le classi della libreria **Java.Security** sia quelle di **NativeDS**, permettendo di eseguire calcoli nativi per parametri come **chiave pubblica** e **generatore**, oltre a gestire **firma e verifica**. In alternativa, si possono usare manualmente i sottotipi di **ParametersCalculator** e **ParametersExtractor**, ottenendo una gestione più flessibile delle classi **Java.Security**.

Quando si vuole avviare un'applicazione con **NativeDS** utilizzando **parametri standard**, è necessario definire i valori della **chiave privata** e del **file da firmare**, entrambi in formato byte array, oltre a un'istanza di **OperationsManager**.

Questa configurazione non richiede di specificare i parametri, poiché è possibile utilizzare la classe **KeyPairGenerator** genererà parametri casuali con le configurazioni di default definiti dalla libreria **Java.Security**, in conformità con gli standard **FIPS**.

Ma se si desidera utilizzare chiavi di dimensione differente, è possibile utilizzare il metodo `initialize` inserendo come parametro il valore in bit desiderato della dimensione del parametro **p**. In questo modo la dimensione degli altri parametri verrà adattata alla nuova dimensione di **p** e i valori verranno generati sulla base di queste nuove configurazioni.

Se invece si preferisce utilizzare **parametri personalizzati**, occorre fornire una specifica, come **DSAParameterSpec**, con tutti i parametri definiti (eccetto il **generatore**, che verrà calcolato successivamente).

Di seguito, vengono illustrati esempi per l'uso di **OperationsManager** con parametri personalizzati e standard:

Parametri personalizzati:

```
OperationsManager oM = new OperationsManager(KeyPairGeneratorAlgorithmName,
                                             hashAlgorithmName);
AlgorithmParameterSpec dsaParams = new DSAParameterSpec(pValue, qValue, null);

dsaParams = oM.calculateGValueAndUpdateParamSpec(dsaParams);
KeyPair keyPair = oM.calculateKeyPair(privateKeyByteValue, dsaParams);

oM.signFile(fileToSignByteValue, keyPair.getPrivate());
```

Parametri standard:

```
OperationsManager oM = new OperationsManager(KeyPairGeneratorAlgorithmName,
                                             hashAlgorithmName);

KeyPair keyPair = KeyPairGenerator.getInstance(KeyPairGeneratorAlgorithmName)
                    .generateKeyPair();

oM.signFile(fileToSignByteValue, keyPair.getPrivate());
```

4.2 Oryx Embedded (C)

Per le implementazioni in **C** mi sono servito delle librerie della **Oryx Embedded**.

Tale scelta è dovuta al fatto che questa azienda è incentrata sulla produzione di librerie mirate ad un vario ambiente **firmware**, permettendo quindi una più agevole migrazione del codice tra vari **ambienti embedded** e **microcontrollori**, oltre che permettere implementazioni in un ambiente **software**.

Le librerie della **Oryx Embedded** sono progettate per essere leggere ed accessibili per dispositivi con poca memoria.

La libreria sembra però mancare di alcune implementazioni relative al calcolo di alcuni **valori** necessari per la generazione della firma.

Probabilmente però ciò è dovuto al fatto che i **valori** in questione sono onerosi da calcolare e sono solitamente forniti al programma come **valori precalcolati**.

4.2.1 Modifiche apportate

Volendo sviluppare un **progetto** completo, capace di calcolare o estrarre casualmente tutti i **valori** necessari, ho deciso di compensare tali mancanze sia usufruendo di implementazioni aggiuntive fornite dalla **Oryx Embedded** (come **yarrow.h**) sia implementando io stesso delle funzioni all'interno dei file della libreria (ogni modifica è stata adeguatamente commentata per evitare equivoci con la versione originale).

Le funzioni da me create sono state implementate per supportare solamente gli algoritmi utilizzati come **SHA256** e non altre versioni degli stessi.

Pertanto in alcuni casi estranei all'esempio proposto nei **main** potrebbero causare il lancio di un errore **ERROR_NOT_IMPLEMENTED**.

4.2.2 Libreria CycloneCrypto

La **CycloneCrypto** è la libreria principale del **progetto**, la quale contiene le implementazioni e le **struct** utilizzate per adempiere alle funzionalità della **firma digitale**, oltre ad altre funzionalità che non verranno menzionate in questo documento.

Link per la documentazione ufficiale della libreria:

https://www.oryx-embedded.com/doc/dir_1702080616e5f4ed1265447cf141dd85.html

Files mpi

Data la necessità di lavorare con **valori** ben più grandi di quelli rappresentabili dalle regolari variabili a **32 bit**, per la realizzazione di implementazioni crittografiche e simili vengono utilizzate

strutture come le **MPI (Multiple precision integer)**.

Tale struttura e le operazioni delle **MPI** sono definite nel pacchetto **cycloneCrypto/mpi**.

La **struct** è definita nel seguente modo:

```
typedef struct
{
    int_t sign;
    uint_t size;
#ifdef (CRYPTO_STATIC_MEM_SUPPORT == DISABLED)
    uint_t *data;
#else
    uint_t data[MPI_MAX_INT_SIZE];
#endif
} Mpi;
```

L'implementazione nel **progetto** prevede di avere il **supporto della memoria statica disabilitato** in quanto è stato pensato per un ambiente software che accetta dati di dimensione variabile anziché registri di dimensione fissa.

Le **MPI** sono progettate per rappresentare **valori** notevolmente grandi sfruttando la concatenazione dei dati contenuti nell'array **data**, gestendoli come una **sequenza continua di bit**, mentre le variabili **sign** e **size** vengono rispettivamente utilizzate per definire **segno** e **dimensione del valore**.

Ad esempio, se un **MPI** ha la variabile **size** impostata a **2**, significa che deve contenere un valore rappresentato da due **uint_t (unsigned int da 32bit)**, avente quindi un **valore** compreso nell'intervallo **[2³³-1, 2⁶⁴-1]**.

Per rappresentare tali **valori**, è necessario utilizzare più **bit** di quelli disponibili nei tipi di variabile standard.

Questo implica la necessità di utilizzare più elementi dell'array **data** per rappresentare un singolo valore. Ad esempio, concatenando i bit di **data[0]** e **data[1]** si ottiene un valore da **64 bit**.

Di default, il valore **size** può arrivare fino a **128**, permettendo alle strutture **MPI** di gestire numeri fino a **4096 bit**. Questo significa che possono rappresentare **numeri estremamente grandi**, che possono superare l'ordine di grandezza di **10¹²⁰⁰**, ideali per applicazioni nei modelli di sicurezza più avanzati.

Il file **mpi.c** implementa tutte le operazioni basilari (lineari e in modulo) tra **grandi valori binari**. Queste operazioni possono essere facilmente estese per supportare calcoli più complessi.

Le **struct Mpi** necessitano di essere inizializzate con la funzione **mpilnit(Mpi *r)**, la quale inizializza la variabile **Mpi** impostando i valori **sign = 1**, **size = 0** e **data = NULL**. Poi la memoria allocata deve essere liberata con la funzione **mpiFree(Mpi *r)**.

Files dsa

Il file **dsa.h** è il file che definisce le funzioni indispensabili per eseguire la **firma digitale DSA** e tutte le **struct** utilizzate dall'algoritmo, definito nel pacchetto **cycloneCrypto/pkc** insieme ad altri algoritmi di crittografia a chiave pubblica.

Le implementazioni fornite dal file **dsa.c** sono perfettamente funzionanti, però non supportano né la **generazione nativa delle chiavi** né del **valore g**, costringendomi quindi ad implementarle io stesso aggiungendo le funzioni necessarie all'interno dei files.

funzioni di inizializzazione:

Dato che ognuna delle **struct** definite dal file **dsa.h** utilizzano variabili **Mpi** per rappresentare i valori numerici, è necessario definire una funzione di inizializzazione per ogni **struct**, nelle quali vengono inizializzati i valori **Mpi**.

Tali funzioni hanno delle denominazioni che iniziano con "**dsaInit-**" seguito dal nome della struct che viene inizializzata.

Le struct allocabili da questo file sono **DsaDomainParameters**, **DsaPublicKey**, **DsaPrivateKey** e **DsaSignature**.

Ognuna di queste struct è composta da delle variabili **Mpi**:

- DsaDomainParameters** contiene i valori **p**, **q**, e **g**
- DsaPublicKey** e **DsaPrivateKey** contengono rispettivamente i valori **x** e **y**
- DsaSignature** contiene i valori **r** ed **s**

Funzioni di liberazione della memoria:

Queste funzioni liberano manualmente la memoria deallocando i puntatori forniti, tali funzioni hanno delle denominazioni che iniziano con "**dsaFree-**" seguito dal nome della struct che viene inizializzata.

Le struct deallocabili da questo file sono **DsaDomainParameters**, **DsaPublicKey**, **DsaPrivateKey** e **DsaSignature**.

funzioni di conversione dei dati:

Queste funzioni vengono utilizzate per **serializzare una firma in un buffer**, ovvero una **sequenza continua di byte**.

Serializzare un dato significa convertire tutte le informazioni contenute nella struct **DsaSignature** in un formato che può essere facilmente **memorizzato o trasmesso**.

La funzione **dsaWriteSignature** converte una **DsaSignature** in un array di byte, mentre la funzione **dsaReadSignature** inverte l'operazione.

Il parametro **length**, richiesto da entrambe le funzioni, fa riferimento alla **lunghezza prevista del buffer espressa in byte**, la quale può essere equivalente alla lunghezza del parametro **q**.

funzioni principali:

Queste funzioni sono quelle responsabili di svolgere le funzionalità principali del file, ovvero **applicare e verificare la firma**.

La funzione **dsaGenerateSignature** viene utilizzata per applicare la firma e richiede molti parametri per operare:

-**contesto**: I primi due parametri sono **PrngAlgo**, fornito come struct costante dal file **yarrow.c**, e un **contesto prng (pseudo-random number generator)** di tipo non definito ma, se si sta utilizzando **Yarrow**, esso dovrà essere una struct **YarrowContext** contenente un **seed**.

Purtroppo i files **yarrow** non forniscono un'implementazione per generare un **seed**, quindi ho implementato all'interno di **yarrow** il metodo **yarrowSetSimpleTimePRSeed**.

Per **inizializzare il generatore yarrow** è necessario a sua volta un valore casuale, ritrovandomi dunque in una situazione paradossale. Quindi ho dovuto servirmi del metodo nativo **rand** utilizzando come **seed** la data attuale in **formato timestamp**.

Ho fatto ciò pur essendo consapevole che un seed generato con bassa entropia compromette anche l'entropia di un generatore affidabile come yarrow, dunque riterrei una buona modifica quella di utilizzare fonti più imprevedibili come un sistema di memorizzazione degli I/O collezionati dal dispositivo in un lungo periodo di tempo.

-**parametri DSA**: I tre parametri principali sono il **byte array** del **digest** del file da firmare, la **lunghezza del byte array espressa in byte**, la **DsaPrivateKey** e la firma da applicare **DsaSignature**, inserita nella funzione per essere sovrascritta.

La funzione **dsaVerifySignature**, utilizzata per la verifica firma, richiede soltanto i **parametri DSA** appena elencati.

funzioni da me implementate:

Per permettere il calcolo nativo dei parametri, ho implementato nei files **dsa** le due funzioni **dsaGenerateGValue** e **dsaGeneratePublicKey**.

La funzione **dsaGenerateGValue** richiede la struct **DsaDomainParameters** con i parametri **p** e **q** impostati, inoltre non prevede la generazione casuale di **h**, richiedendo quindi di essere estratta in precedenza.

Una volta in possesso del valore **g** è possibile copiare **DsaDomainParameters** all'interno della **DsaPrivateKey** per poterla utilizzare come parametro nella funzione **dsaGeneratePublicKey**.

Files ecdsa ed ec

Le implementazioni dei files **ecdsa** funzionano esattamente come quelle delle definite nei files **dsa**, variando solo dal tipo di struct utilizzati. Quindi la spiegazione dei files **dsa** precedentemente fornita è valida anche per descrivere le funzioni **ecdsa**.

Per l'algoritmo **ECDSA** sono presenti nella libreria **cycloneCrypto/ecc** anche i files **ec**, i quali implementano il calcolo della chiave pubblica e l'estrazione casuale della chiave.

Purtroppo nonostante l'aggiunta dei file **ec** non viene fornita l'implementazione del calcolo di **G** e, a mia sorpresa, non è risultata presente neanche l'implementazione per il calcolo dell'**ordine del punto G**.

Fortunatamente però il file **ec.c** implementa le funzioni per la somma e il raddoppio dei punti di una curva ellittica tramite l'utilizzo di coordinate proiettive. Ciò, a differenza delle implementazioni della **Java.Security** basate su punti a due coordinate, mi ha permesso di implementare una

funzione per il calcolo dell'**ordine di G** e di finalizzare l'implementazione dell'**algoritmo ECDSA con parametri personalizzati** in C.

funzioni di inizializzazione e liberazione della memoria:

Dato che ognuna delle **struct** definite dal file **ec.h** utilizzano variabili **Mpi** per rappresentare i valori numerici, è necessario definire una funzione di inizializzazione per ogni **struct**, nelle quali vengono inizializzati i valori **Mpi**.

Le funzioni che allocano e liberano la memoria hanno delle denominazioni che iniziano rispettivamente con "ecInit" e "ecFree", seguiti dal nome della **struct** interessata.

Le **struct** definite e gestite dai files **ec** sono **EcDomainParameters**, **EcPoint**, **EcPublicKey** ed **EcPrivateKey**.

Funzioni principali:

Queste sono le funzioni che hanno come responsabilità quella di adempiere alle funzionalità del file, ovvero **generare una coppia di chiavi** e **eseguire operazioni sui punti** della curva ellittica.

La funzione **ecGeneratePublicKey** esegue dei calcoli deterministici, mentre **ecGeneratePrivateKey** richiede un **contesto prng**, spiegato nella sezione **dsa**, in quanto genera una chiave casuale. Di conseguenza la funzione **ecGenerateKeyPair** richiede anchessa il **contesto prng** in quanto esegue le due funzioni di generazione delle chiavi.

Le due funzioni che permettono il calcolo della chiave pubblica, applicazione e verifica della firma sono **ecDouble** ed **ecAdd**, le quali eseguono le operazioni di raddoppio e di somma dei punti della curva ellittica con coordinate proiettive (**x, y, z**).

Funzioni da me Implementate:

Dato che la libreria **CycloneCrypto**, a differenza della **Java.Security**, non permette l'acquisizione di parametri standard precalcolati, implementare tutte le funzioni necessarie per il calcolo nativo dei dati diventa quasi necessario, ma lo avrei fatto comunque.

Le funzioni mancanti da me aggiunte sono **ecGenerateCurvePoint** e **ecCalculatePointOrder**.

La prima funzione calcola il **punto G** data una **coordinata x** mentre la seconda calcola il grado di un dato punto.

Entrambe le funzioni richiedono anche un **EcDomainParameters** come parametro per eseguire i calcoli e prevedono di deallocare la variabile del risultato in caso di fallimento.

La funzione **ecGenerateCurvePoint** in particolare ritorna errore con la stragrande maggioranza delle coordinate x fornite. Questo non è dovuto ad un errore di implementazione, ma al fatto che per ogni curva esistono relativamente pochi punti aventi entrambe le coordinate intere. Questo perché la formula $y^2 = x^3 + ax + b$ spesso genera valori decimali, non validi per la matematica modulare.

La funzione **ecCalculatePointOrder** invece ritorna errore solo se il punto fornito non appartiene alla curva o ci sono altri errori di configurazione dei parametri.

4.2.3 Libreria Common

Questa libreria, ampiamente utilizzata dalle altre librerie della **Oryx Embedded**, fornisce implementazioni generali per la manipolazione dei dati, le operazioni di rete, la crittografia e la gestione degli errori.

Le sue implementazioni sono orientate principalmente ad **ambienti firmware**, offrendo codice altamente **portabile** ma anche **ottimizzato** per vari **processori** e **microcontrollori**.

Il mio **progetto** è stato pensato per funzionare come una **funzione atomica autonoma**, che opera **senza** interagire con altri elementi dell'**ambiente firmware** oltre agli input necessari per avviare l'esecuzione.

Nonostante la libreria **Common** sia stata utilizzata il minimo indispensabile per soddisfare le dipendenze del progetto, essa dovrebbe semplificare la configurazione del codice per renderlo compatibile con specifici **ambienti firmware** (es. **M460**).

Link per la documentazione ufficiale della libreria:

https://www.oryx-embedded.com/doc/dir_bdd9a5d540de89e9fe90efdfc6973a4f.html

4.2.4 Files di configurazione esterni

La libreria che ho scaricato non forniva i due **file di configurazione** necessari e quindi, non sapendo come configurare la libreria, li ho scaricati da dei repository che ho trovato su **GitHub**.

crypto_config.h:

Link per il repository **GitHub**:

https://github.com/steffanw/CycloneTCP/blob/master/demo/atmel/sam3x_ek/web_server_demo/src/crypto_config.h

Il file **crypto_config.h** è un file di configurazione utilizzato per **personalizzare le funzionalità e le impostazioni** della libreria crittografica fornita da **Oryx Embedded** (come parte del pacchetto **CycloneCRYPTO** o altre librerie crittografiche).

La sua funzione principale è consentire all'utente di **abilitare o disabilitare algoritmi crittografici**, funzionalità specifiche e ottimizzazioni a seconda delle esigenze del progetto.

Nel progetto ho **disabilitato il supporto** per tutti gli **algoritmi** e i **microcontrollori** menzionati in questo file di configurazione ad eccezione fatta per gli algoritmi **SHA256** e **AES**.

os_port_config.h:

Link per il repository **GitHub**:

https://github.com/rpc-fw/analyzer/blob/master/thdanalyzer_m0/src/CycloneTCP/config/os_port_config.h

Il file di configurazione **os_port_config.h** permette di **personalizzare la libreria per il sistema operativo in tempo reale** utilizzato.

La sua principale funzione è abilitare l'uso del codice specifico del **RTOS** all'interno della libreria. Aggiungendo questo file mancante e configurandolo correttamente, si permette alla libreria di interagire l'**RTOS** scelto, come **FreeRTOS** o un altro **RTOS** compatibile.

Ma dato che il mio **progetto** le implementazioni sono state pensate per un **ambiente software**, il contenuto di questo file è stato commentato.

4.2.5 Configurazioni per DSA ed ECDSA in C

La libreria **CycloneCrypto** non include la possibilità di ottenere dei parametri standard, portandomi a decidere di includere solamente due implementazioni con **parametri personalizzati**, una per ogni algoritmo di firma digitale implementato.

Configurazioni di avvio

Il linguaggio di programmazione **C**, non essendo un linguaggio di programmazione ad oggetti, non possiede il polimorfismo, però la libreria **CycloneCrypto** ha delle funzioni molto simili tra i due algoritmi. Inoltre ogni funzione ha praticamente lo stesso nome della sua corrispettiva dell'altro algoritmo, mantenendo come unica differenza i prefissi "dsa" ed "ec".

Queste somiglianze permettono una più agevole comparazione delle implementazioni.

Quando si vuole avviare un'applicazione con la libreria **CycloneCrypto**, è innanzitutto necessario inizializzare la struct della specifica dei parametri corretta (**DsaDomainParameter** o **ECDomainParameter**), per poi impostare manualmente i valori dei parametri iniziali.

Se si sta inizializzando la **struct DSA** dovranno essere impostati i valori dei parametri **p** e **q** della struct più un ulteriore parametro casuale **h** compreso tra **1** e **p-1**.

Mentre per una **struct ECDSA** è necessario impostare i valori dei parametri **a**, **b**, **p** e della coordinata **x** del punto **G**.

Esempi di codice:

```
DsaDomainParameters params;
dsaInitDomainParameters(&params);
mpiSetValue(&params.p, DSA_PRIME_VALUE);
mpiSetValue(&params.q, DSA_MODULE);

Mpi h;
mpiInit(&h);
mpiRandRange(&h, &params.p, &yarrowPrngAlgo, &yContext);
```

```
EcDomainParameters params;
ecInitDomainParameters(&params);
mpiSetValue(&params.a, A_VARIABLE);
mpiSetValue(&params.b, B_VARIABLE);
mpiSetValue(&params.p, FIELD_LIMIT);
mpiSetValue(&params.g.x, G_POINT_X_COORDINATE);
```

Se si vuole inizializzare un **context** da utilizzare per una generazione di numeri sicura, è possibile utilizzare **yarrow**, implementato nel pacchetto **rng** della **CycloneCrypto**.

Nei files **yarrow** ho aggiunto la funzione **yarrowSetSimpleTimePRSeed** che permette di inizializzare il **seed del contesto yarrow**, anche se la generazione della mia implementazione non possiede molta entropia.

Una volta impostati i parametri iniziali si prosegue al calcolo del generatore tramite le funzioni **dsaGenerateGValue** per **DSA** e **ecGenerateCurvePoint** per **ECDSA**.

Se si sta utilizzando la **struct ECDSA** è necessario utilizzare anche la funzione **ecCalculatePointOrder**.

Per la generazione della **chiave pubblica** è necessario inizializzare la **chiave privata** impostare il valore manualmente e utilizzare la funzione **dsaGeneratePublicKey** o **ecGeneratePublicKey**.

Se si sta utilizzando la chiave privata **DsaPrivateKey** è anche necessario copiare i parametri dentro la chiave tramite la funzione **dsaDomainParametersCopy(&privK.params, ¶ms)**.

Ora che i parametri e le chiavi sono stati calcolati è necessario proseguire con la realizzazione della firma.

La firma viene eseguita dalle funzioni **dsaGenerateSignature** ed **ecdsaGenerateSignature**, le quali necessitano del **digest del file da firmare**.

Per avere il **digest** è necessario usare un algoritmo di cifratura e la libreria CycloneCrypto ne fornisce alcuni, tra cui **SHA256**.

Quindi è possibile utilizzare la funzione **sha256Compute(FILE_TO_SIGN, sizeof(digest), digest)**, dove il **digest** è un **array di uint8_t** (praticamente un byte array) di lunghezza pari alla costante **SHA256_DIGEST_SIZE** (equivalente a 32) in riferimento alla lunghezza in byte. Se si utilizza uno SHA di diversa lunghezza si dovrà cambiare lunghezza del digest di conseguenza.

Per verificare la firma si utilizzano le funzioni **ecdsaVerifySignature** e **dsaVerifySignature**.

Serializzazione della firma

Una volta generata la firma è probabile che si voglia avere la possibilità di cifrarla prima di archivarla o inviarla tramite il web. Però proteggere una struttura dati tramite cifratura è un'operazione più complessa della cifratura di una stringa o di un byte array.

La libreria **CycloneCrypto** fornisce delle funzioni apposite per i suoi algoritmi, come **ecdsaWriteSignature** e **dsaWriteSignature**, che convertono in maniera reversibile la firma da **struct** in un **array di uint_8**, e le funzioni **ecdsaWriteSignature** e **dsaWriteSignature**, che riconvertono la firma da **array** a **struct**.

La dimensione in byte della **firma serializzata** dovrebbe essere uguale a quella dell'**ordine q** utilizzato dall'algoritmo che ha generato la firma.

Successivamente, avendo la **firma convertita in un array di byte**, è possibile procedere agevolmente con una funzione di cifratura (es **sha256Compute**) per mettere in maggiore sicurezza le informazioni.

Di seguito viene riportato come serializzare e deserializzare una firma **DSA**:

```
uint8_t buffer[sizeof(&params.q.data)];
size_t bufferSize = sizeof(buffer);
ecdsaWriteSignature(&sign, buffer, &bufferSize);
//invia il buffer della firma
//-----
//riceve il buffer e lo riconverte
DsaSignature receivedSignature;
ecdsaInitSignature(&receivedSignature);
ecdsaReadSignature(buffer, bufferSize, &receivedSignature);
```

Conclusione

L'obiettivo di questa tesi è dimostrare come l'**ECDSA** possa rappresentare un'alternativa più efficiente per il futuro rispetto a **DSA** ed **RSA**, soprattutto in vista delle dimensioni sempre maggiori delle chiavi richieste dagli algoritmi tradizionali. Rendendo **ECDSA** un ottimo algoritmo per il futuro in termini di risparmio sul consumo di risorse.

In conclusione, lo studio conferma che l'**ECDSA**, a parità di sicurezza fornita, richiede meno risorse computazionali di **RSA**.

Tuttavia, l'effettivo impatto di questa tesi potrebbe limitarsi a fungere come strumento didattico per comprendere il funzionamento dei due algoritmi a livello matematico e come proposta di implementazione per progetti privati.

In questo contesto, la tesi si prefigge di fungere come documento didascalico per permettere una comprensione dei principi matematici alla base degli algoritmi **DSA** ed **ECDSA**, proponendo una serie di spiegazioni matematiche atte a coprire tutti gli argomenti necessari per iniziare lo studio delle due firme elettroniche presentate e fornendo il codice sorgente per agevolare la comprensione degli algoritmi.

Bibliografia

- [1] Niels Ferguson – Bruce Schneier – Tadayoshi Kohno, “Il manuale della Crittografia”, Apogeo, 2011
- [2] Bruce Schneier, “Applied cryptography : protocols, algorithms, and source code in C”, New York: Wiley, 1996
- [3] National Institute of Standards and Technology, “FIPS 186-4, Digital Signature Standard (DSS), U.S. Department of Commerce/National Institute of Standards and Technology”, Federal Information Processing Standards Publication, July 2013.
- [4] Chuck Easttom, “Modern Cryptography Applied Mathematics for Encryption and Information Security Second Edition”, Springer, 2022
- [5] Marco Cocilova (Marclova), “CryptographyImplementations” (progetto affiliato alla tesi), GitHub, <https://github.com/Marclova/CryptographyImplementations.git>, 2024
- [6] Oryx Embedded, “CycloneCrypto”, sito ufficiale, <https://www.oryx-embedded.com/products/CycloneCRYPTO.html>, 2024
- [7] Oracle Corporation, “Java.Security”, documentazione ufficiale, <https://docs.oracle.com/javase/8/docs/api/java/security/package-summary.html>, 2024
- [8] AdoptOpenJDK, “openjdk-jdk11”, GitHub, <https://github.com/AdoptOpenJDK/openjdk-jdk11.git>, 2018